Desarrollo de un prototipo web

ElOnceIdeal

Alumnos :
Aldrey Muiño, Darío
Boullón Espiño, Ángela

7 de agosto de 2025

ElOnceIdeal

Índice

1. Introducción **3**

2. Organización del contenido **3**
2.1. Arquitectura de la información . . . 3
2.2. Mapa de navegación . . . 4

3. Diseño de interfaces **6**
3.1. Sketch . . . 6
3.2. Wireframes . . . 7
3.3. Mockups . . . 11
3.4. StoryBoard . . . 17

4. Jerarquía de archivos **18**

5. Consideraciones **19**

6. Mapa de etiquetas HTML **20**
6.1. Página principal . . . 20
6.1.1. Barra de navegación . . . 21
6.1.2. Secciones de contenido principal . . . 22
6.1.3. Pie de página . . . 22
6.2. Página de ligas . . . 22
6.2.1. Barra de navegación principal . . . 24
6.2.2. Barra de navegación secundaria. Selector de ligas . . . 24
6.2.3. Información general sobre la liga . . . 24
6.2.4. Sección de clasificación . . . 24
6.3. Página de calendario de partidos . . . 24
6.3.1. Barra de navegación . . . 26
6.3.2. Carrusel de selección de fecha . . . 26
6.3.3. Partidos del día seleccionado . . . 26
6.4. Página de partido . . . 26
6.4.1. Barra de navegación . . . 28
6.4.2. Contenido principal del partido . . . 28
6.4.3. Ficha del partido . . . 28
6.5. Página de noticias . . . 28
6.5.1. Barra de navegación . . . 29
6.5.2. Secciones de contenido . . . 29

7. CSS **30**
7.1. Estilo global . . . 30
7.2. Efectos y Responsividad . . . 30
7.3. Flexible Grids: Página principal . . . 31

7.3.1. Responsividad 31
7.4. CSS Multicol: Página de noticias 33
7.4.1. Responsividad 33
7.5. Flex Container: Página de calendario 34
7.5.1. Responsividad 35
7.6. CSS Grid: Página de partido 37
7.6.1. Responsividad 37
7.7. Librería Bootstrap: Página de ligas 39
7.7.1. Responsividad 39

8. JavaScript **41**
8.1. Semana 1: DOM 41
8.1.1. Noticias.js 41
8.1.2. Calendario.js 42
8.2. Semana 2: JQuery y JES6 45
8.2.1. JQuery (noticas.js) 45
8.2.2. JES6 (calendario.js) 46
8.3. Semana 3: Carga de contenido 47
8.3.1. Carga de XML con fetch (Noticias.js) 47
8.3.2. Carga de JSON con XMLHttpRequest (Calendario.js) 49

1. Introducción

Se solicita la creación de una página web dedicada al fútbol denominada **ElOnceIdeal**, en alusión al número de jugadores de un mismo equipo que coinciden en el campo durante un partido del susodicho deporte. El contenido del sitio estará exclusivamente centrado en las cinco principales ligas europeas: LaLiga (España), Premier League (Inglaterra), Bundesliga (Alemania), Serie A (Italia) y Ligue 1 (Francia).

Hablar de ligas implica ser consciente de cuáles son los equipos que en ellas participan. Además, al tratar la información de un club, resulta fundamental conocer a sus jugadores. Asimismo, los campeonatos no existirían sin los partidos que se organizan según un calendario. Finalmente, las noticias y el contenido multimedia sobre las ligas son elementos de gran interés para el usuario final. Toda esta información relevante para nuestro sitio web es recogida en la figura 1

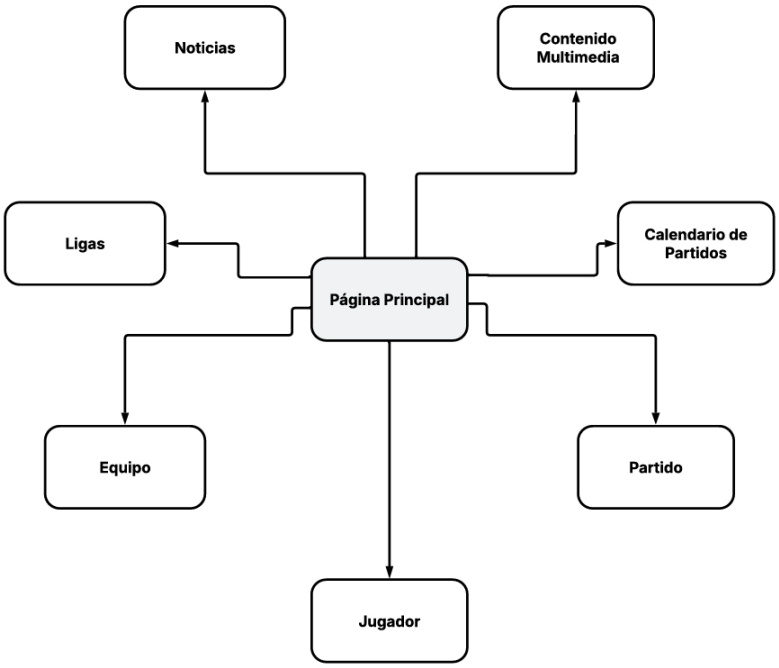


Figura 1: Inventario de contenido

2. Organización del contenido

2.1. Arquitectura de la información

Toda la información del sitio se organiza jerárquicamente partiendo de la página principal, tal y como se muestra en la figura 2. Según vista dicha figura:

- Los equipos se sitúan un nivel por debajo de las ligas, dado que estas permiten

organizar a los clubes por países y establecer un orden entre ellos mediante una clasificación.

- Los jugadores se sitúan un nivel por debajo de los equipos, pues son los profesionales las piezas que componen la plantilla de los clubes.
- Los partidos se sitúan un nivel por debajo del calendario, puesto que este permite organizar los encuentros según una fecha y hora.

Se incluyen en la arquitectura de la información subtemas relevantes; véase Galería de imágenes y Resúmenes de partidos bajo Contenido multimedia en la figura 2.

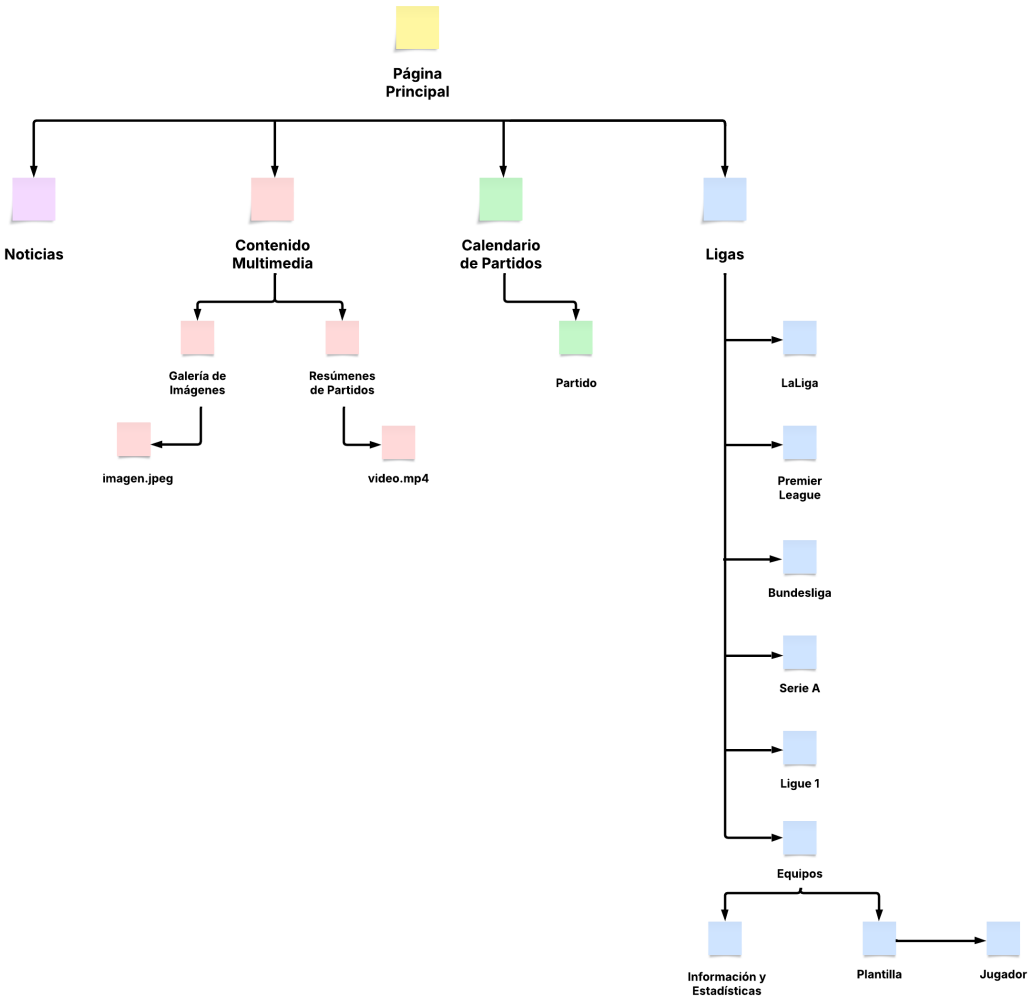


Figura 2: Arquitectura de la información

2.2. Mapa de navegación

En esta fase, el contenido de la página web previamente estructurado se concreta en las interfaces que compondrán el sitio. También se determina la navegación entre dichas

interfaces. Son ocho las interfaces escogidas:

1. Página principal.
2. Noticias.
3. Contenido multimedia.
4. Ligas.
5. Calendario de partidos.
6. Equipo.
7. Jugador.
8. Partido.

En la figura 3 puede verse el contenido del que dispone cada una de las interfaces mencionadas, además de las rutas de navegación que se establecen entre ellas. Las interfaces de Noticias, Contenido multimedia, Ligas y Calendario de partidos son siempre accesibles desde la barra de navegación. Luego, se identifican dos rutas de navegación importantes:

1. Calendario de partidos $\overset{a}{\Longrightarrow}$ Partido
2. Ligas $\overset{a}{\Longrightarrow}$ Equipo $\overset{a}{\Longrightarrow}$ Jugador

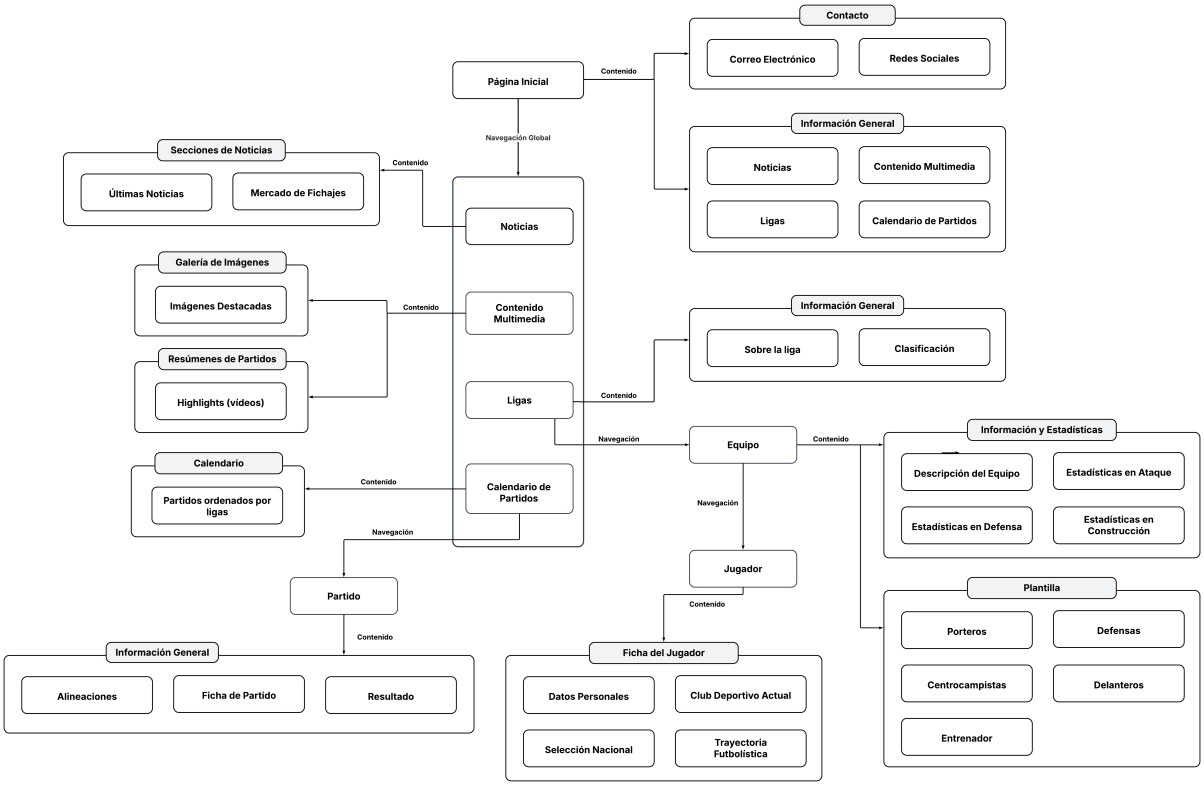


Figura 3: Mapa de navegación

3. Diseño de interfaces

3.1. Sketch

Se recogen los bocetos iniciales que esbozan las ideas para el diseño de cada una de las interfaces. En pro de la organización se agrupan en tres figuras siguiendo un criterio de navegación entre interfaces:

1. La figura 4 muestra el bosquejo de las interfaces que participan en la navegación global, incluyéndose: Página principal, Noticias, Contenido multimedia, Ligas y Calendario de partidos.

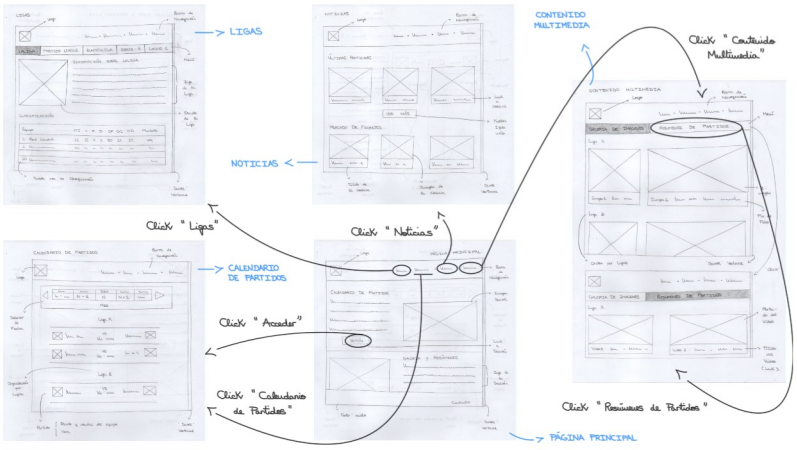


Figura 4: Sketchs desde la página principal

2. La figura 5 muestra el bosquejo para las interfaces: Ligas, Equipo y Jugador.

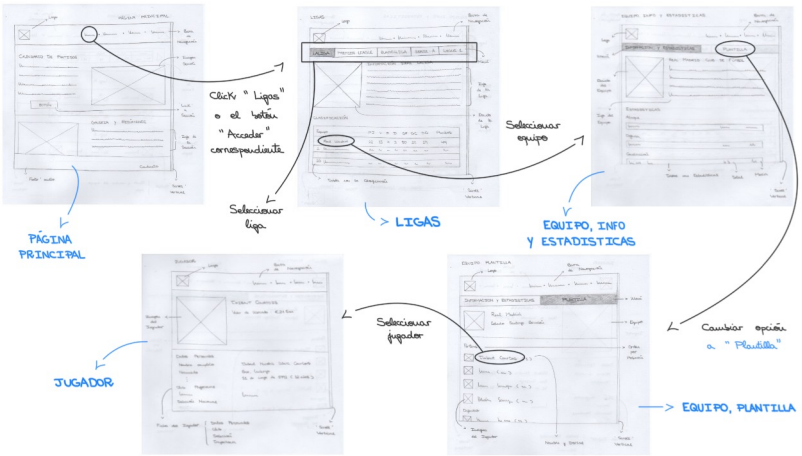


Figura 5: Sketchs de Ligas, Equipo y Jugador

3. La figura 6 muestra el bosquejo para las interfaces: Calendario de partidos y Partido.

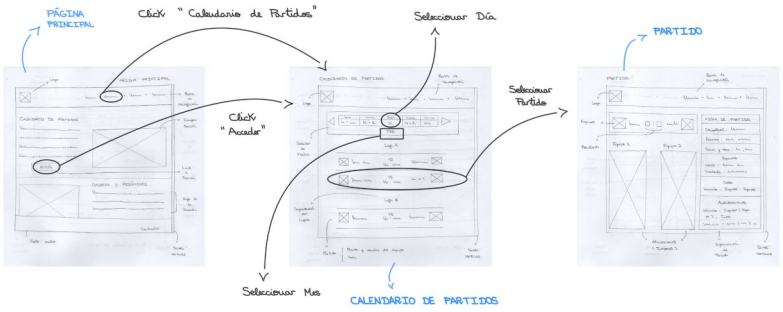


Figura 6: Sketchs de Calendario de Partidos y Partido

3.2. Wireframes

Sobre los wireframes se determina con precisión la posición y dimensiones de los elementos que componen cada interfaz. Se emplea un diseño en 12 columnas, donde el espacio horizontal se divide en 12 porciones de igual longitud. Para tomar decisiones referentes a la dimensión de varios elementos o espacios se empleará la proporción áurea. Decisiones comunes a todos los wireframes:

- Partiendo de un tamaño de la fuente de 12px para los cuerpos de texto. Aplicando la proporción áurea, obtenemos para los títulos y/o encabezados un tamaño de 19px ($12 \cdot 1{,}618 \approx 19$). Excepcionalmente para la interfaz de Jugador (figura 10), se cuenta con un título aún mayor, su tamaño será de 31px ($19 \cdot 1{,}618 \approx 31$).

- La mayoría de espacios que se dividen en dos columnas siguen una proporción 5-7, nuevamente por la regla áurea. Es la división del espacio de doce columnas que más se aproxima a dicha proporción.

- La regla áurea se aplica a las proporciones de muchas de las imágenes presentes en la web. Véanse las ilustraciones de sección en la interfaz de la Página principal (figura 7) o las alineaciones en la interfaz de Partido (figura 12).

Divisiones del espacio horizontal más específicas:

- La interfaz del Calendario de partidos (figura 11) emplea un espacio de 10 columnas para el selector de fechas, dejando una columna blanca a izquierda y otra a derecha. La lista de los partidos emplea un espacio de 8 columnas, dejando 2 columnas blancas a izquierda y derecha. De esta forma se reducen las distracciones en los laterales y se centra la atención en una lista concisa de elementos centrales.

- La interfaz de Noticias (figura 13) presenta hasta cuatro noticias por fila, esto es, cada elemento ocupa 3 columnas.

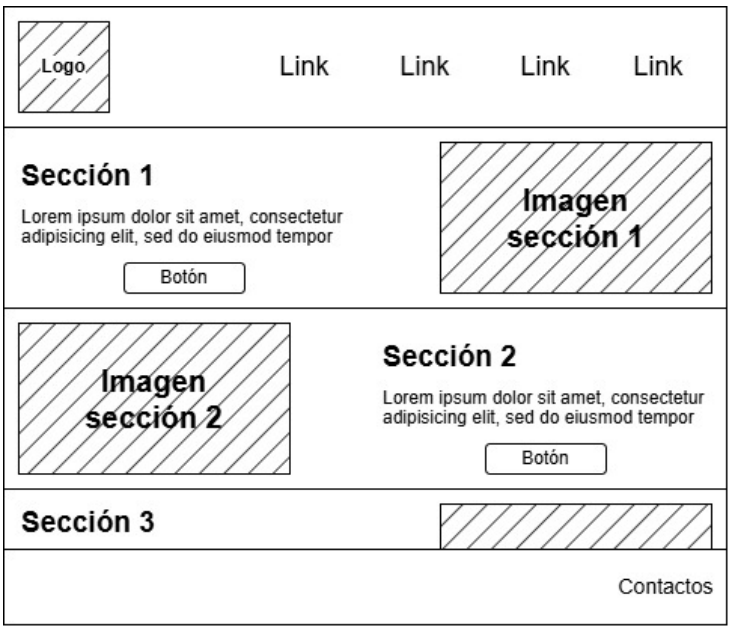


Figura 7: Wireframe: Página Principal

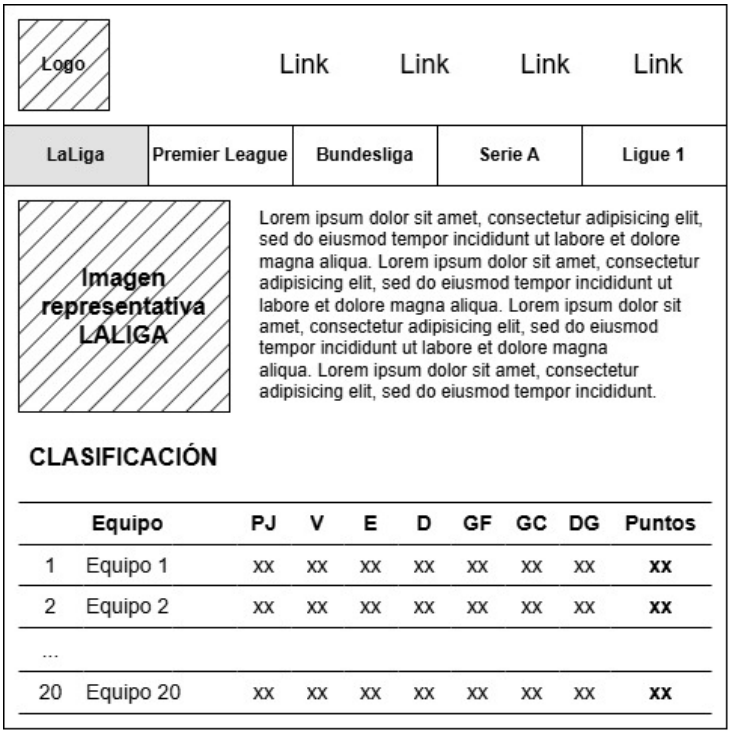


Figura 8: Wireframe: Ligas

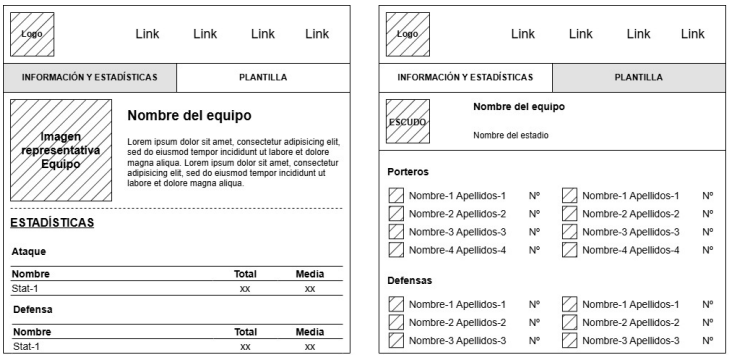


Figura 9: Wireframe: Equipo

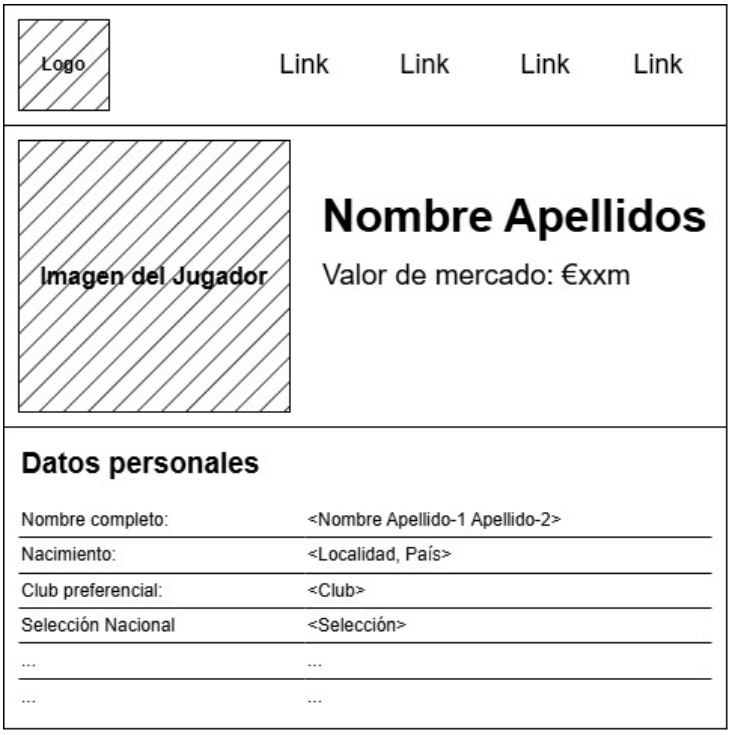


Figura 10: Wireframe: Jugador

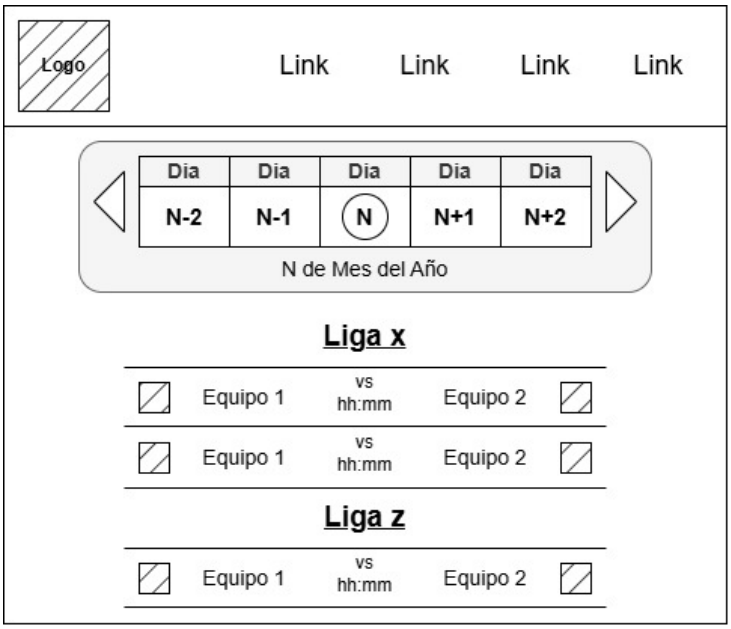


Figura 11: Wireframe: Calendario de partidos

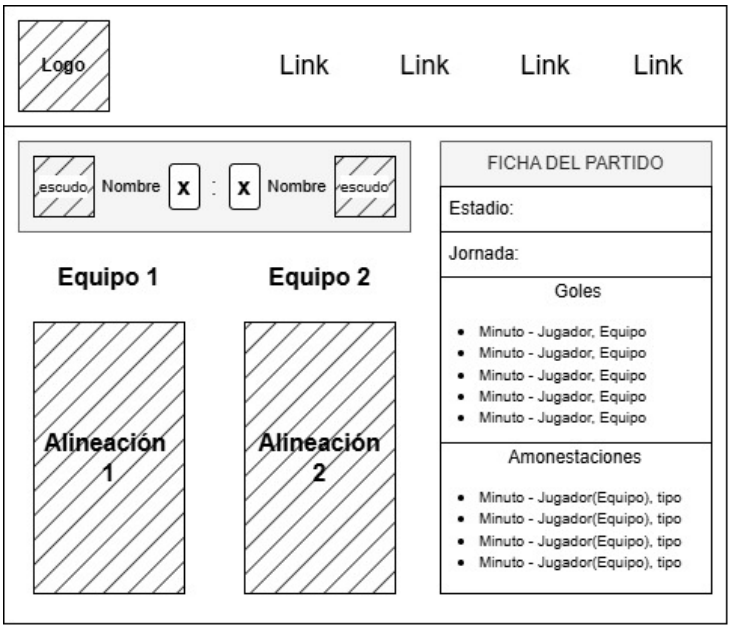


Figura 12: Wireframe: Partido

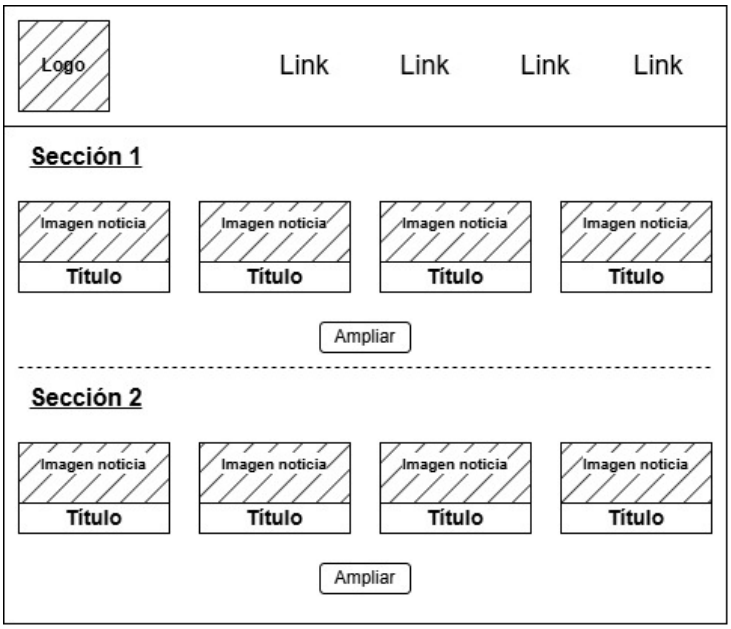


Figura 13: Wireframe: Noticias

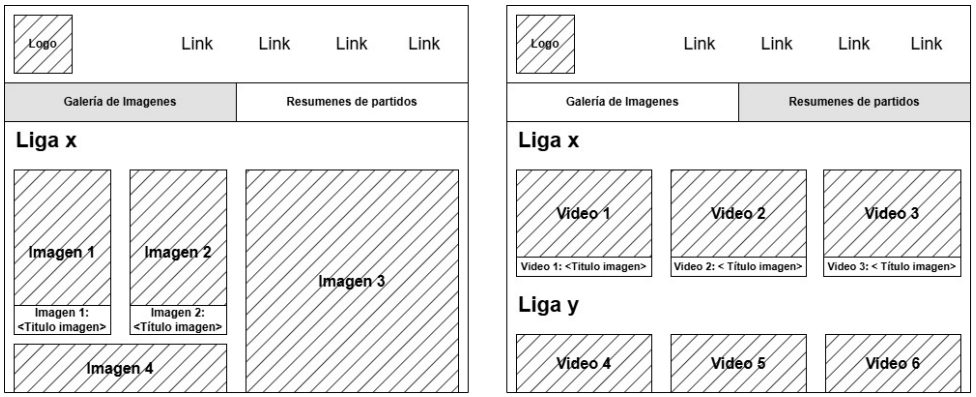


Figura 14: Wireframe: Contenido Multimedia

3.3. Mockups

Una vez finalizada la definición de la estructura base a través de los wireframes, se ha procedido a la generación de mockups que representen visualmente cada una de las secciones del sitio. Esta etapa permite plasmar con mayor grado de precisión la apariencia del producto final, incluyéndose aspectos como la paleta de colores y la tipografía. El resultado es recogido en las figuras 15-24.

Para llevar a cabo la tarea, se ha recurrido al empleo de Canva como instrumento de diseño. Ciertas limitaciones en la interfaz de la herramienta han impedido que las proporciones áureas establecidas previamente fuesen respetadas con exactitud. No obstante,

sí serán aplicadas con corrección en la implementación del producto en la nube, asegurándose de esta manera el mantenimiento de fidelidad entre el plan original y la conclusión.

En cuanto a la identidad visual del proyecto, el logo creado para la web representa a una persona jugando al fútbol, lo que refuerza la temática del sitio de manera clara y directa.

Respecto a la paleta cromática, esta está compuesta por azul (#004aad) y blanco (#ffffff). El azul ha sido escogido por su capacidad para transmitir confianza y profesionalidad, fortaleciéndose de esta manera el carácter informativo del que se pretende dotar a las páginas. A diferencia de una paleta en tonos rojos, que enfatizaría la pasión y un enfoque más fanático, el azul genera una sensación de credibilidad y seriedad. En contraste, el fondo blanco aporta limpieza y facilita la legibilidad del contenido, prometiendo una experiencia visual equilibrada y accesible.

Finalmente, para la redacción se ha empleado Montserrat, una tipografía moderna y versátil que garantiza una lectura cómoda al mismo tiempo que proporciona una estética técnica. Además, su diseño geométrico confirma coherencia y claridad en la presentación de la información.

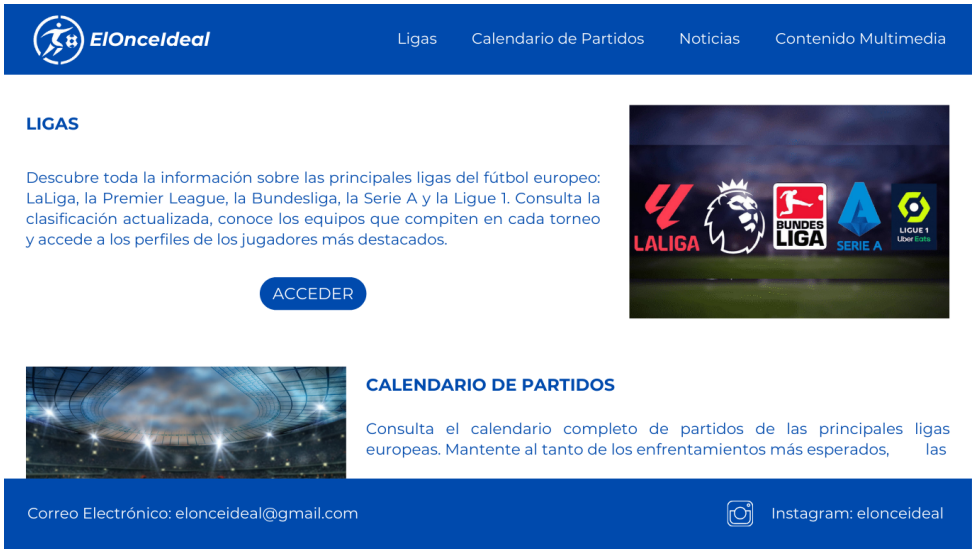


Figura 15: Mockup: Página Principal



Figura 16: Mockup: Ligas

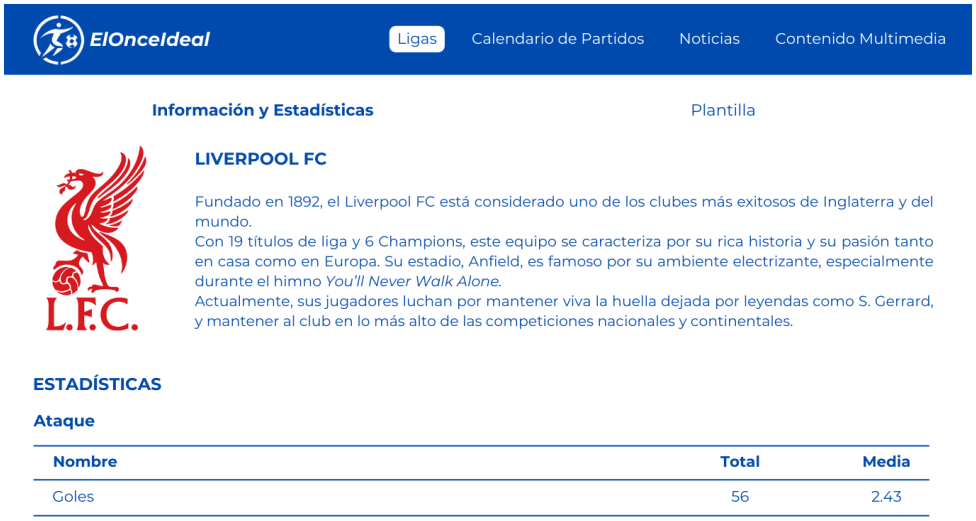


Figura 17: Mockup: Equipos (Información y Estadísticas)



Figura 18: Mockup: Equipos (Plantilla)

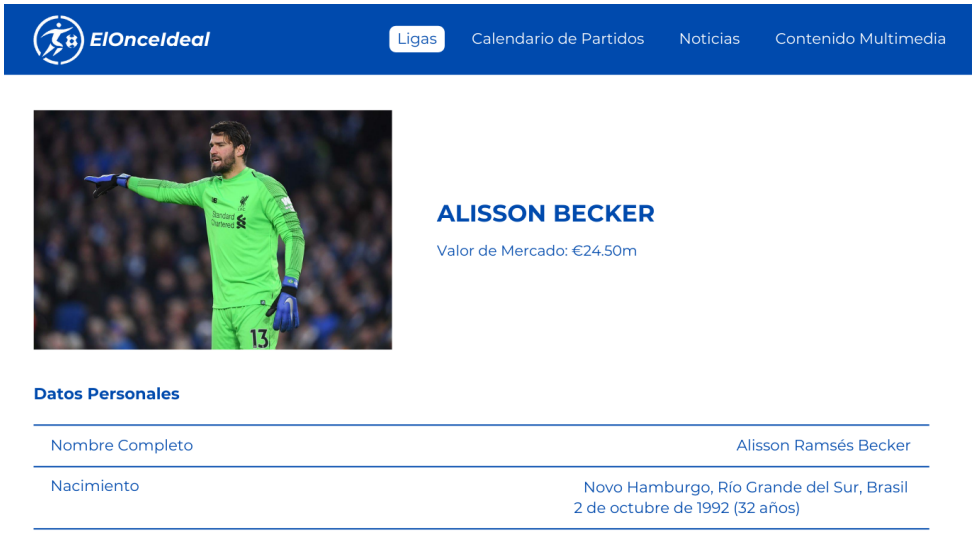


Figura 19: Mockup: Jugador



Figura 20: Mockup: Calendario de partidos



Figura 21: Mockup: Partido

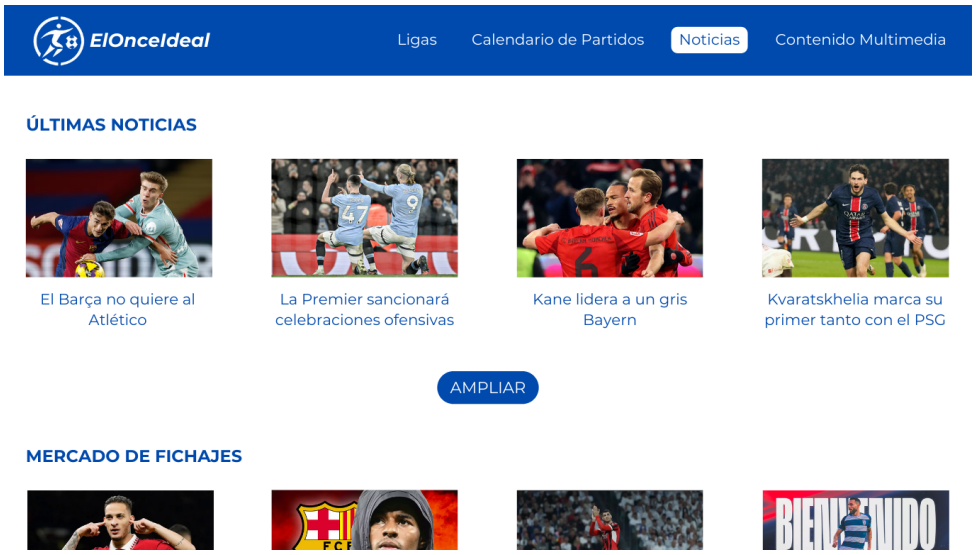


Figura 22: Mockup: Noticias

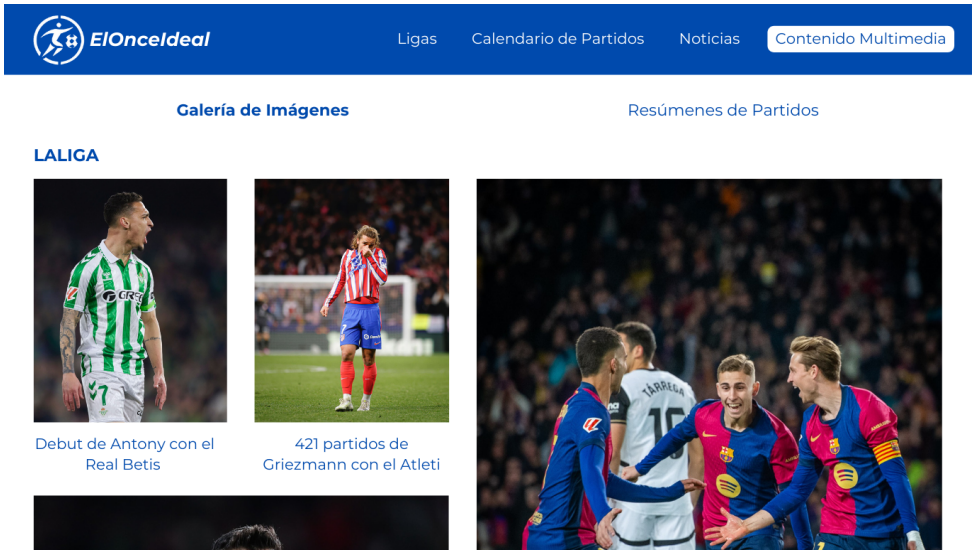


Figura 23: Mockup: Contenido mutltimedia (Galería de imágenes)

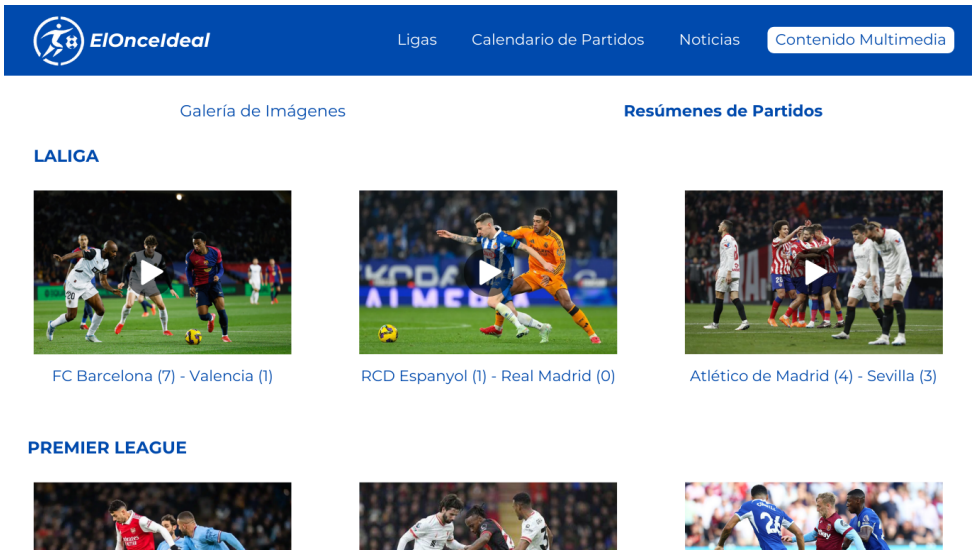


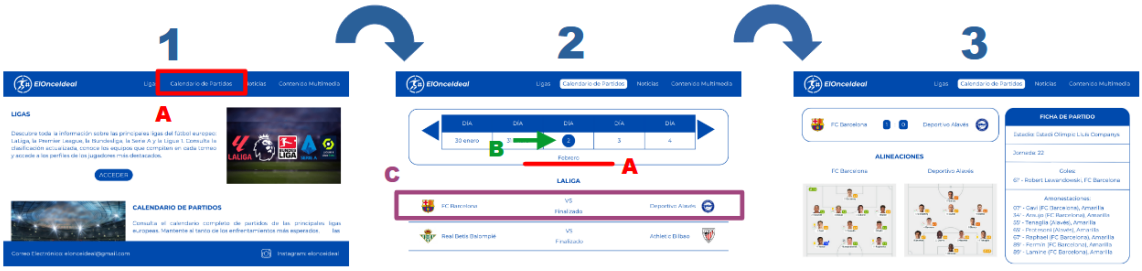
Figura 24: Mockup: Contenido multimedia (Resúmenes de partidos)

3.4. StoryBoard

Finalizada la creación de mockups, en las figuras 25 y 26 se explican a través de storyboards los dos flujos de navegación más destacados. Estes se corresponden con el acceso a las páginas Jugador y Partido, respectivamente. El resto de interacciones se consideran intuitivas.



Figura 25: StoryBoard: Navegación hacia Jugador



Cuando el usuario accede a la web, lo primero que se muestra en pantalla es la Página Principal. A partir de ella se puede acceder al resto de secciones.

Para visualizar la información sobre un partido concreto, primero pulsamos la opción "Calendario de Partidos" de la barra de navegación (A).

Luego, insertamos la fecha (A)(B) cuya programación nos interesa conocer. En pantalla se muestra una lista de todos los encuentros fijados para ese día, ordenados por ligas.

Finalmente, 'clickamos' encima del partido que nos interesa (C).

Como resultado, en la pantalla se muestra información detallada sobre el encuentro seleccionado: resultado, alineaciones y ficha de partido.

Figura 26: StoryBoard: Navegación hacia Partido

4. Jerarquía de archivos

En la figura 27 se muestra la organización de los archivos del proyecto. En el primer nivel de la jerarquía encontramos el archivo html de la página principal (index.html), la documentación del proyecto y los directorios:

- **src**: Contiene los archivos html de la página web. Ficheros relevantes:
  1. CalendarioDePartidos.html
  2. Noticias.html
  3. Partidos/Partido.html
  4. **Ligas**/ Contiene una subcarpeta por liga, la cual cada una de ellas tiene el archivo html correspondiente y un subdirectorio de Equipos. En el directorio de equipos se encuentran las carpetas de los equipos que contienen dos html: equipo y plantilla; además del subdirectoriocon los html correspondientes a los jugadores de dichos equipos.
- **Imágenes**/: Contiene las imagenes empleadas en las distintas interfaces y secciones del sitio web. Se encuentran separadas por secciones.
- **css**/: Contiene las hojas de estilo de las páginas.
- **js**/: Contiene los scripts del sitio web.

- **data/**: Contiene información que será cargada en algunas de las páginas del sitio web. Concretamente noticias.xml y partidos.json.
- **Montserrat**. Contiene las fuentes montserrat empleadas en el sitio web.

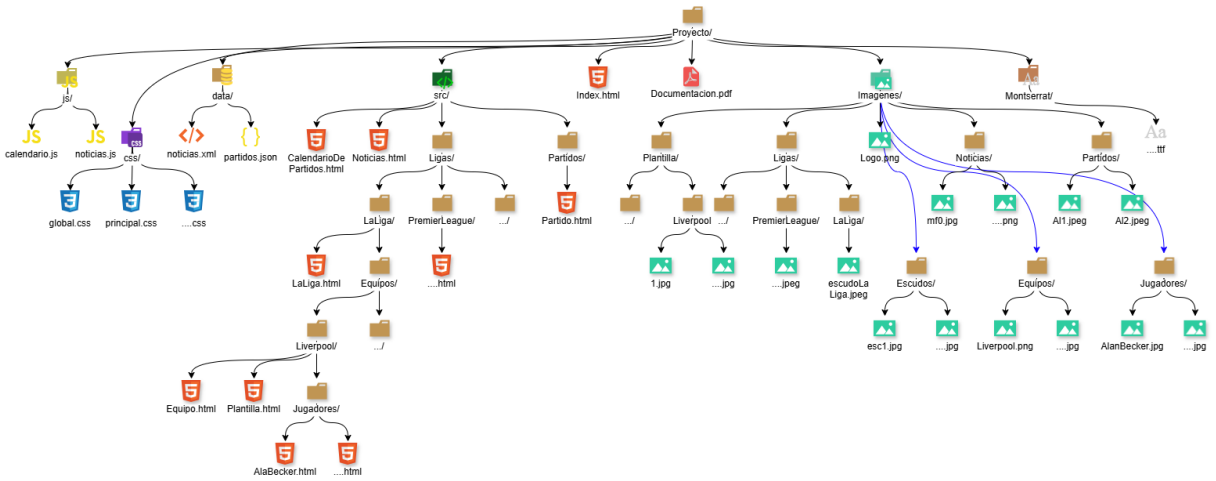


Figura 27: Jerarquía de archivos

5. Consideraciones

Desde la primera entrega del proyecto, se han producido algunos ajustes en el alcance y la estructura del sitio web:

- **Eliminación de la página Contenido Multimedia:** Se ha decidido prescindir de esta sección, por lo que no se ha implementado ninguna página dedicada a dicho contenido.
- **Páginas representativas únicas:** En lugar de generar una página específica para caa jugador, equipo, plantilla o partido, se ha optado por crear una única página representativa para cada tipo de contenido. Estas páginas actúan como ejemplo del diseño y estructura que tendría cada una en su versión completa del sitio. En concreto:
  - La página del equipo y plantilla representa al Liverpool FC.
  - La página de partido corresponde al enfrentamiento entre el FC Barcelona y el Deportivo Alavés.
  - La página de jugador presenta información sobre Alisson Becker.
- **Enlaces redirigidos a las páginas representativas:** Todos los enlaces dentro del sitio que, en una implementación completa, deberían conducir a una página específica de equipo, jugador, plantilla o partido, han sido redirigidos a la página representativa correspondiente.

6. Mapa de etiquetas HTML

A continuación, se detallan los mapas de etiquetas empleados para diseñar algunas de las páginas más relativas del sitio web.

6.1. Página principal

La estructura de la página principal ha sido desarrollada siguiendo una jerarquía lógica y semántica de etiquetas HTML5. De esta forma, se permite una correcta organización del contenido desde el punto de la presentación, la accesibilidad y el posicionamiento en los diferentes motores de búsqueda. La imagen asociada a este esquema es la figura 28.

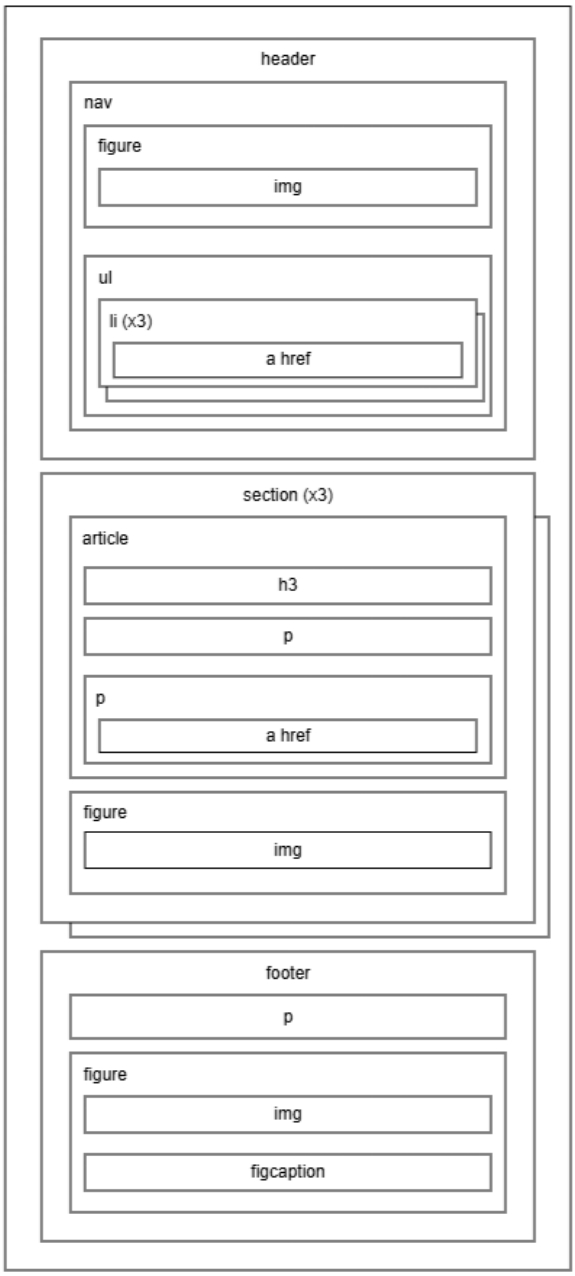


Figura 28: Mapa de etiquetas de index.html

6.1.1. Barra de navegación

La parte superior de la página está ocupada por una barra que permite al usuario acceder de forma rápida a las diferentes secciones del sitio. Esta sección se implementa con la etiqueta <header>que, a su vez, contiene un bloque de navegación principal definido con la etiqueta <nav>. Dentro de este segundo componente se incluyen:

- Un <figure>, donde se incorpora el logotipo de la página mediante una imagen (<img>).
- Una lista desordenada, <ul>, que contiene tres ítems (<li>). Cada uno de ellos está asociado a un enlace <a href>que redirige a una de las secciones principales: Ligas, Calendario de partidos y Noticias.

6.1.2. Secciones de contenido principal

La zona central de la página presenta tres bloques destacados, correspondientes a las secciones principales del sitio web. Cada uno de ellos se define mediante una etiqueta <section>compuesta por las siguientes piezas:

- Un <article>, que agrupa el contenido textual informativo. Este artículo tiene un encabezado de nivel tres con el nombre de la sección (<h3>), y un párrafo con una breve descripción del contenido disponible en ella (<p>).
- Una figura, <figure>, con una imagen representativa del contenido del apartado.

6.1.3. Pie de página

La parte inferior de la página incluye un bloque informativo implementado mediante la etiqueta <footer>. En esta zona se ofrece información de contacto básica. Su estructura es la siguiente:

- Un párrafo <p>, cuyo contenido se corresponde con el correo electrónico de la empresa.
- Una figura <figure>, compuesta por el icono de la red social Instagram (<img>), y el nombre de usuario con el que la empresa se ha registrado en ella (<figcaption>).

6.2. Página de ligas

La página de ligas está diseñada para presentar información específica sobre las principales ligas de fútbol (LaLiga, Premier League, Bundesliga, Serie A y Ligue 1), permitiendo al usuario seleccionar la competición de su interés y consultar tanto una descripción general de la misma como la clasificación actual. El mapa de etiquetas asociado a este apartado se recoge en la figura 29.

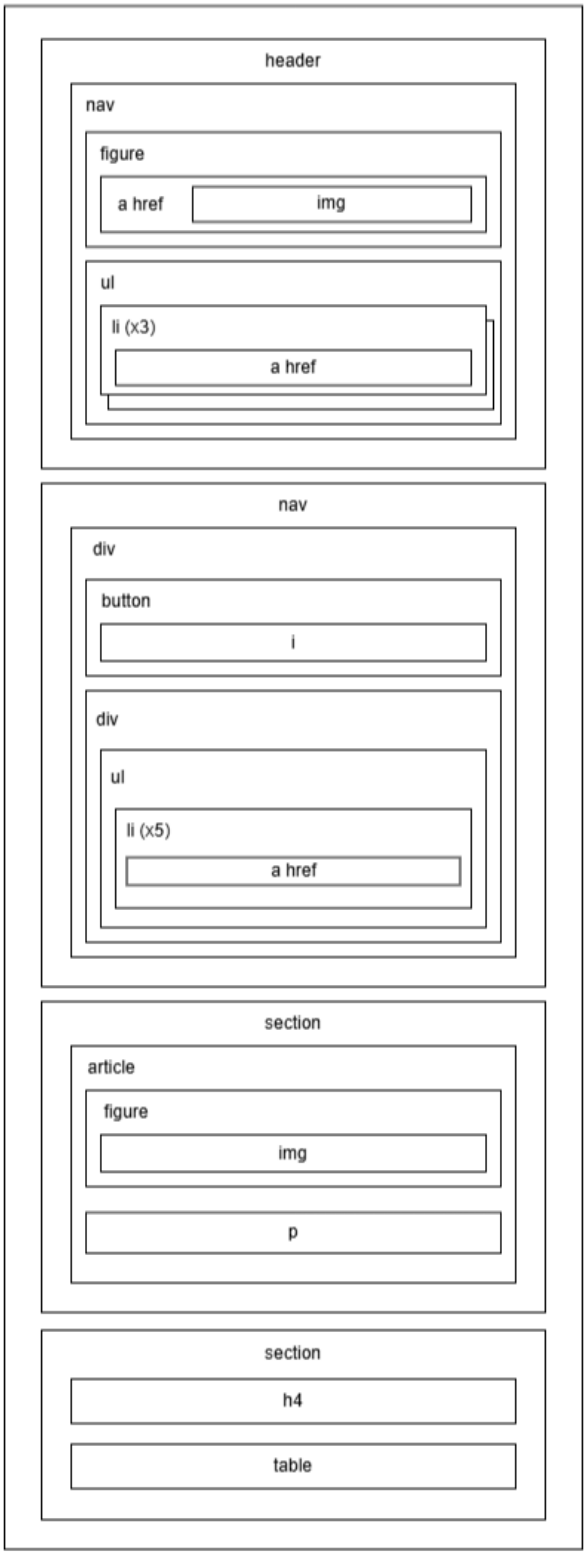


Figura 29: Mapa de etiquetas de ligas.html

6.2.1. Barra de navegación principal

Al igual que en la página principal, la cabecera del sitio se implementa mediante la etiqueta <header>, que contiene un bloque de navegación definido con <nav>. Este incluye:

- Un <figure>con un enlace <a href>que rodea el logotipo del sitio web (<img>) y permite regresar a index.html.
- Una lista de navegación (<ul>) compuesta por tres elementos <li>, cada uno con un enlace <a href>que permite al usuario consultar en qué página se encuentra y acceder a otras, como Calendario o Noticias.

6.2.2. Barra de navegación secundaria. Selector de ligas

Justo debajo de la navegación principal, se ha incorporado una barra secundaria que ofrece al usuario la posibilidad de elegir entre las cinco principales ligas europeas (<nav>). Su estructura es la siguiente:

- Un botón (<button>), que permite añadir responsividad a la página.
- Un segundo contenedor , que aloja una lista formada por cinco ítems <li>. Cada uno de ellos cuenta con un enlace <a href>correspondiente a una liga concreta.

6.2.3. Información general sobre la liga

A continuación se presenta una primera sección informativa en la que se muestra una introducción a la liga seleccionada (<section>). Está compuesta por un <article>que contiene una figura con el escudo de la competición (<figure>), y un párrafo con una breve descripción de la misma (<p>).

6.2.4. Sección de clasificación

Seguidamente, se presenta la clasificación de los equipos estructurada dentro de una segunda sección (<section>). Este apartado incluye un subtítulo introducitorio (<h4>), y la propia tabla de datos con la clasificación (<table>).

6.3. Página de calendario de partidos

La página CalendarioDePartidos.html tiene como objetivo principal mostrar al usuario los partidos programados para una fecha concreta, organizados por liga. El mapa de etiquetas asociado a este apartado se muestra en la figura 30.

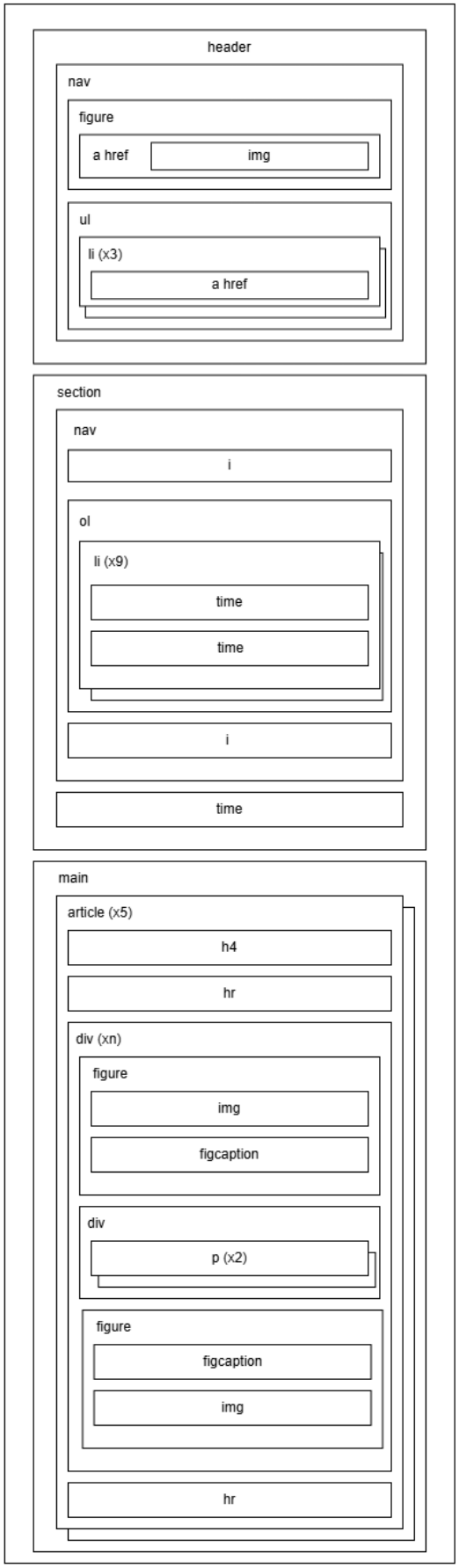


Figura 30: Mapa de etiquetas de CalendarioDePartidos.html

6.3.1. Barra de navegación

Como en el resto de las páginas, se incluye en la cabecera una barra de navegación principal <header>con su correspondiente estructura interna <nav>, que permite acceder a las distintas secciones del stio web.

6.3.2. Carrusel de selección de fecha

A continuación, se presenta una sección <section>destinada a la selección de fechas a través de un carrusel horizontal. La nevagación dentro de este elemento se implementa con una etiqueta <nav>, que incluye:

- Dos iconos <i>, uno a cada lado, que premiten desplazarse hacia atrás o hacia delante en la selección.
- Un contenedor que alberga una lista de elementos <li>, cada uno representando un día del calendario. Cada uno de estos ítems contiene un <time>visual con el día de la semana, y otro con el número del día del mes.

Debajo del carrusel, se muestra con una etiqueta <time>la fecha seleccionada por el usuario, que determina los partidos mostrados en el contenido principal.

6.3.3. Partidos del día seleccionado

La sección central de la página se implementa con la etiqueta <main>, donde se agrupan todos los encuentros del día ordenados por liga. Para establecer una separación clara entre competiciones, se utiliza un componente <article>con las siguientes características:

- Cada artículo comienza con un encabezado <h4>que indica el nombre de la liga.
- Se añade una línea divisoria con la etiqueta <hr>antes y después de los partidos para mejorar la organización visual.
- A continuación, se listan los n partidos de esa liga dentro de un contenedor . Cada encuentro está compuesto por:
  - Un <figure>con el escudo (<img>) y el nombre del equipo local (<figcaption>).
  - Un bloque central ( ) con dos párrafos (<p>): "vsz el estado del partido.
  - Un segundo <figure>con la información del equipo visitante (<img>y <figcaption>).

6.4. Página de partido

La página Partido.html proporciona una visión detallada de un partido de fútbol concreto, incluyendo el resultado, las alineaciones de ambos equipos y una ficha técnica con información adicional sobre el encuentro. El mapa de etiquetas correspondiente se muestra en la figura 31.

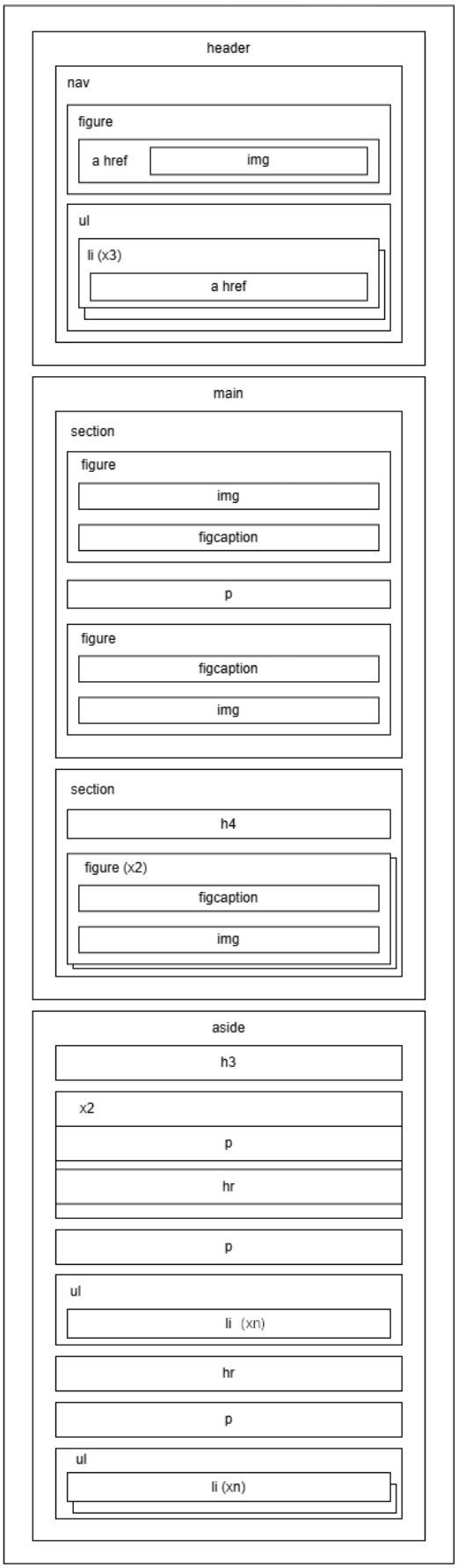


Figura 31: Mapa de etiquetas de Partido.html

6.4.1. Barra de navegación

Como en el resto de las páginas, se incluye en la cabecera una barra de navegación principal <header>con su correspondiente estructura interna <nav>, que permite acceder a las distintas secciones del stio web.

6.4.2. Contenido principal del partido

El cuerpo principal de la página se estructura dentro de la etiqueta <main>, que contiene dos secciones diferenciadas:

a) Resultado del partido: Este apartado muestra de forma visual y clara el marcador final. Se utiliza una etiqueta <section>para contener el resultado, y dos elementos <figure>para representar a los equipos local y visitante. Para cada uno de los participantes se ofrece su escudo (<img>) y nombre (<figcaption>). Además, entre ambos <figure>se encuentra un párrafo, <p>, con el resultado del encuentro.

b) Alineaciones: Esta sección también está contenida en una etiqueta <section>. Comienza con un encabezado <h4>que anuncia su contenido, y presenta un elemento <figure>por equipo. Dentro de cada uno de estes se incluye un <figcaption>con el nombre del participante, y un <img>con la representación gráfica de los jugadores titulares sobre el campo.

6.4.3. Ficha del partido

Finalmente, se incluye una sección con información adicional implementada mediante la etiqueta <aside>. Comienza con un título <h3>que introduce el bloque. A continuación, se muestran varios párrafos <p>con información relevante sobre el encuentro, separados por líneas divisorias <hr>. Finalmente, en la parte inferior se incluyen dos listas no ordenadas (<ul>), una para los goles (n) y otra para las amonestaciones (n).

6.5. Página de noticias

La estructura de la página Noticias.html está diseñada para mostrar al usuario información de actualidad futbolística, divida en dos bloques principales: Últimas noticias y Mercado de fichajes. La organización semántica del contenido facilita su comprensión visual y jerárquica. El mapa de etiquetas asociado a este apartado del sitio se muestra en la figura 32.

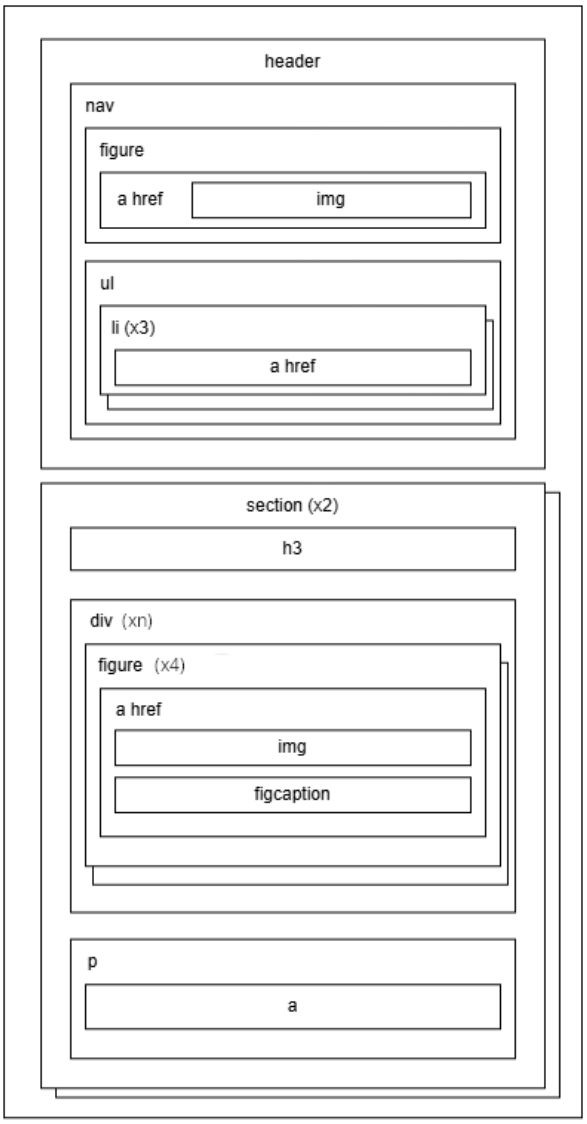


Figura 32: Mapa de etiquetas de Noticias.html

6.5.1. Barra de navegación

Como en el resto de las páginas, se incluye en la cabecera una barra de navegación principal <header>con su correspondiente estructura interna <nav>, que permite acceder a las distintas secciones del stio web.

6.5.2. Secciones de contenido

A continuación, se presentan dos secciones <section>: una para las Últimas noticias, y otra para el Mercado de fichajes. Ambas mantienen una estructura común:

- Un encabezado <h3>que introduce el tipo de contenido mostrado en la sección.

- n contenedores , cada uno de ellos agrupando 4 noticias. Estas se representan individualmente mediante un <figure>que contiene un enlace, <a href>, con una imagen representiva y el titular.
- Al final, un párrafo <p>que contiene un enlace <a>para mostrar más noticias relacionadas con esa categoría.

7. CSS

7.1. Estilo global

Para mantener una coherencia visual y una estructura uniforme en todo el sitio web, se han definido una serie de hojas de estilo CSS específicas. Estas hojas controlan tanto el diseño global como elementos concretos recurrentes en las distintas páginas del sitio. A continuación, se detalla la función de cada uno de estos archivos:

- **global.css:** Esta hoja define el estilo principal del sitio web. Se encarga del formato de la cabecera (<header>) y la barra de navegación (<nav>), aplicando un color de fondo azul corporativo (#004aad), márgenes, paddings y flotado de elementos. También establece la tipografía general del sitio mediante la fuente personalizada Montserrat, integrada con @font-face. Además, se da estilo a los enlaces de navegación y botones, definiendo su comportamiento al pasar el cursor (hover). El pie de página (<footer>) también se estructura desde este archivo, con distribución en flexbox y colores corporativos.
- **hr.css:** Este archivo contiene las reglas para personalizar las líneas horizontales (<hr>), utilizadas como elementos de separación estética entre secciones del contenido.
- **subnav_style.css:** Define la apariencia de la barra de navegación secundaria que puede aparecer en ciertas páginas. Ajusta sus colores y su espaciado, garantizando que se mantenga visualmente integrada con el resto del sitio.
- **table_style.css:** Aplica estilos a las tablas presentes en el sitio, incluyendo bordes y colores.

Con estos documentos, se logra una apariencia uniforme y profesional, facilitando la experiencia del usuario y asegurando la responsividad y accesibilidad del contenido.

7.2. Efectos y Responsividad

El diseño visual del sitio se apoya en distintas tecnologías de maquetación proporcionadas por CSS, combinando enfoques clásicos y modernos para una mejor organización y una experiencia de usuario adaptable a distintos dispositivos. Estas son: Flexible Grids, CSS Multicol, Flex Container, CSS Grid y la librería Bootstrap. A continuación, se detallan los

casos concretos en los que se ha aplicado cada técnica, incluyendo el esquema de cajas asociado y capturas que evidencien los efectos de responsividad.

7.3. Flexible Grids: Página principal

En la página principal se ha aplicado una maquetación basada en Flexible Grids, permitiendo que los elementos se distribuyan proporcionalmente dentro de una rejilla fluida. Esta técnica asegura que el diseño se mantenga ordenado, adaptándose a distintas resoluciones sin desbordamientos ni solapamientos. El esquema correspondiente se recoge en la figura 33.

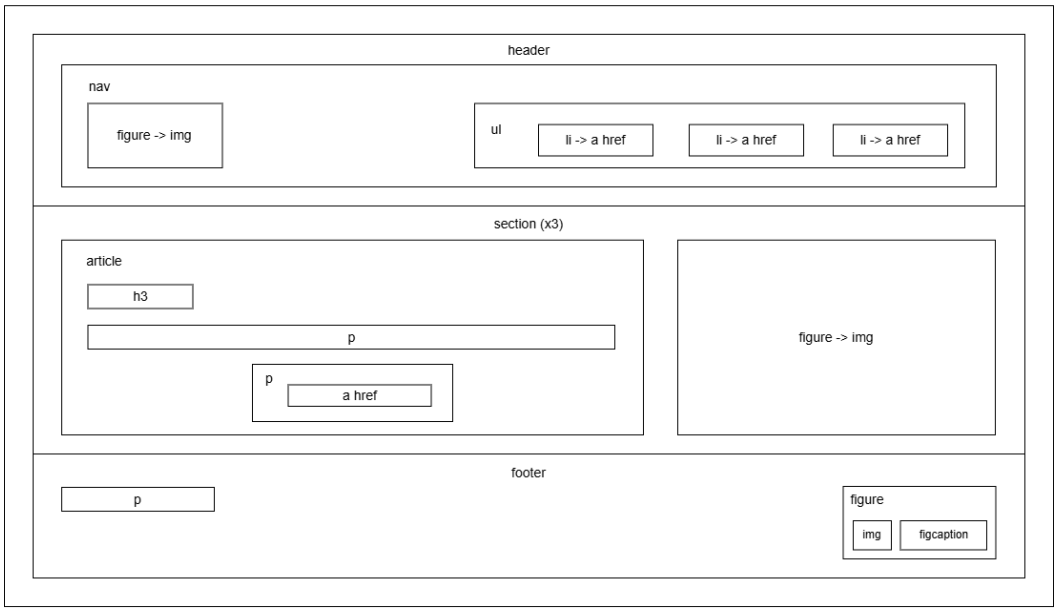


Figura 33: Esquema CSS de la página principal

7.3.1. Responsividad

A través de una media query con punto de ruptura en 768px, los elementos se adaptan para ocupar el 100 % del ancho disponible en pantallas pequeñas, eliminando los flotandos y centrando el contenido (figuras 34 y 35)

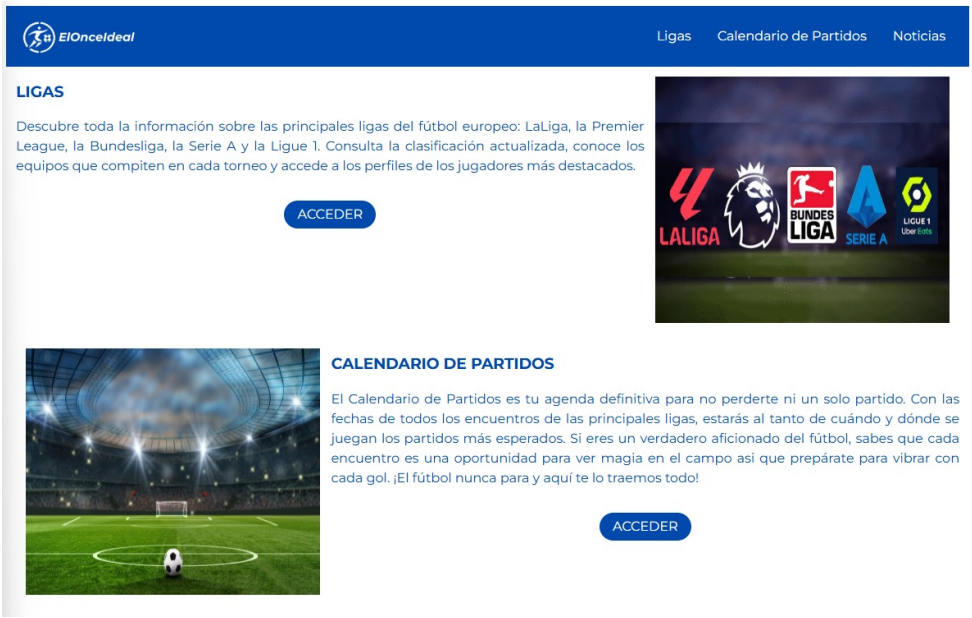


Figura 34: Responsividad: Página principal vista en ordenador

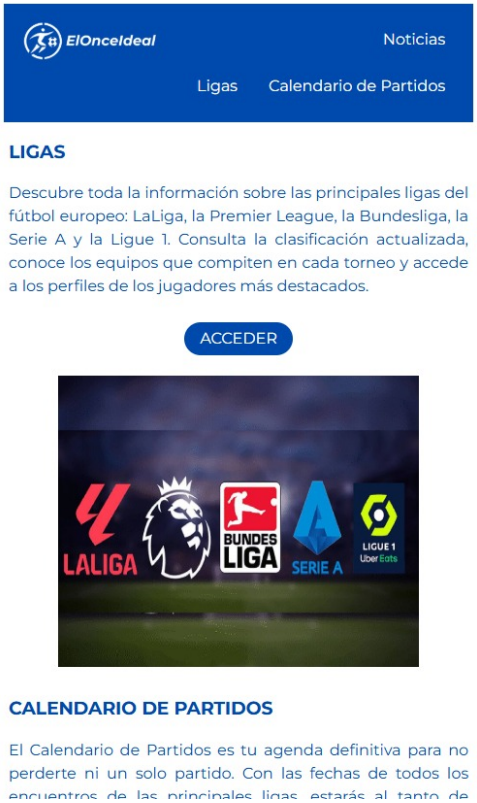


Figura 35: Responsividad: Página principal vista en móvil

7.4. CSS Multicol: Página de noticias

En la página de noticias, se emplea la propiedad column-count para presentar múltiples noticias en formato de columnas. La técnica de maquetación utilizada es, por lo tanto, CSS Multicol. El esquema CSS asociado a este apartado se muestra en la figura 36.

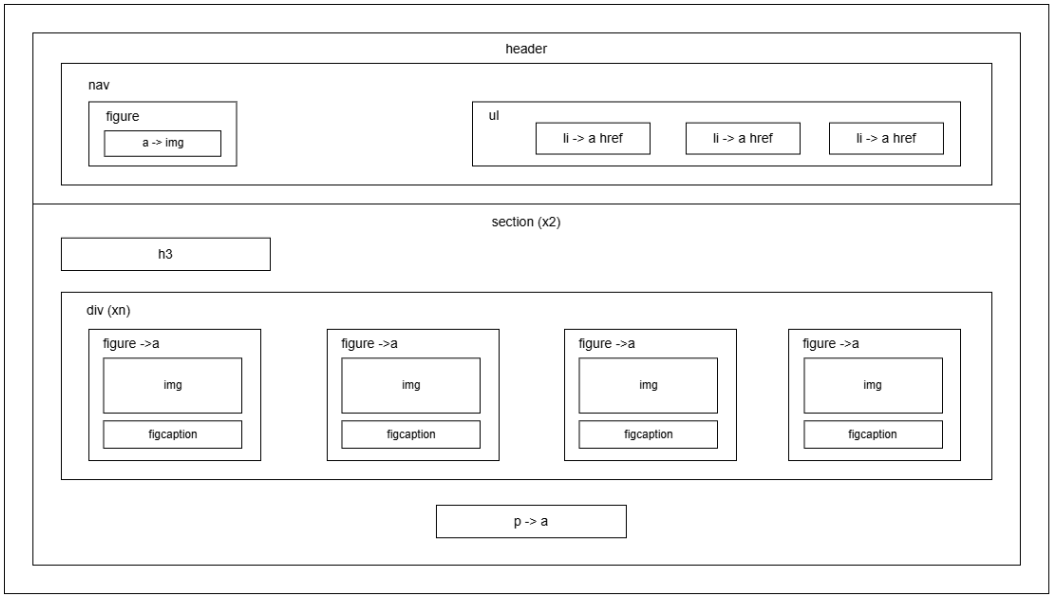


Figura 36: Esquema CSS de la página de noticias

7.4.1. Responsividad

Se emplea una media query con punto de ruptura en 768px, en la cual se fuerza el uso de una única columna para mejorar la lectura en dispositivos móviles (figuras 37 y 38).

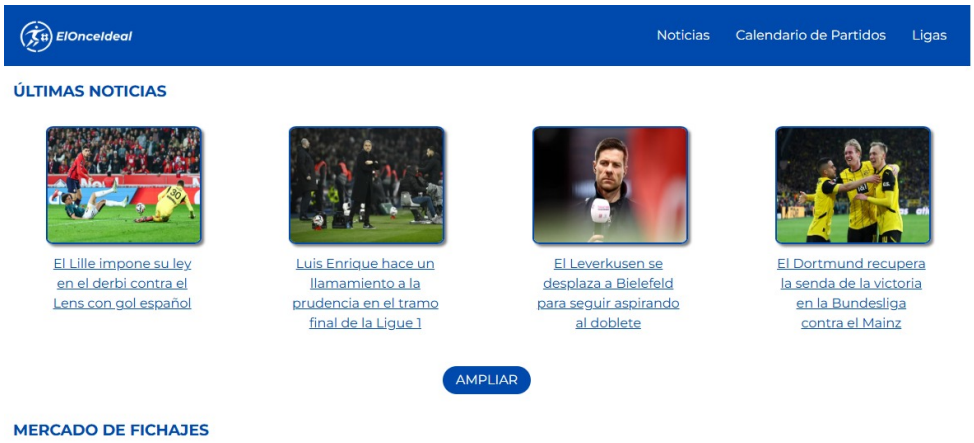


Figura 37: Responsividad: Página de noticias vista en ordenador

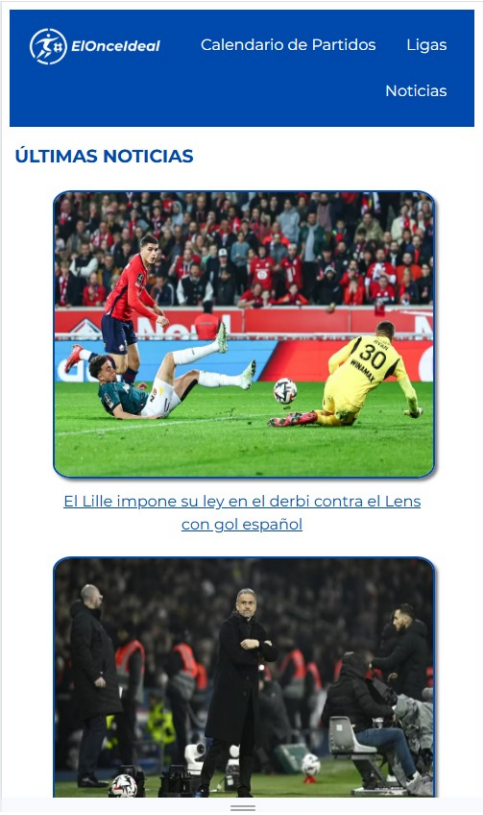


Figura 38: Responsividad: Página de noticias vista en móvil

7.5. Flex Container: Página de calendario

En la página de calendario se utiliza Flex Container para gestionar el carrusel de fechas y la presentación de partidos. El esquema correspondiente se representa en la figura 39.

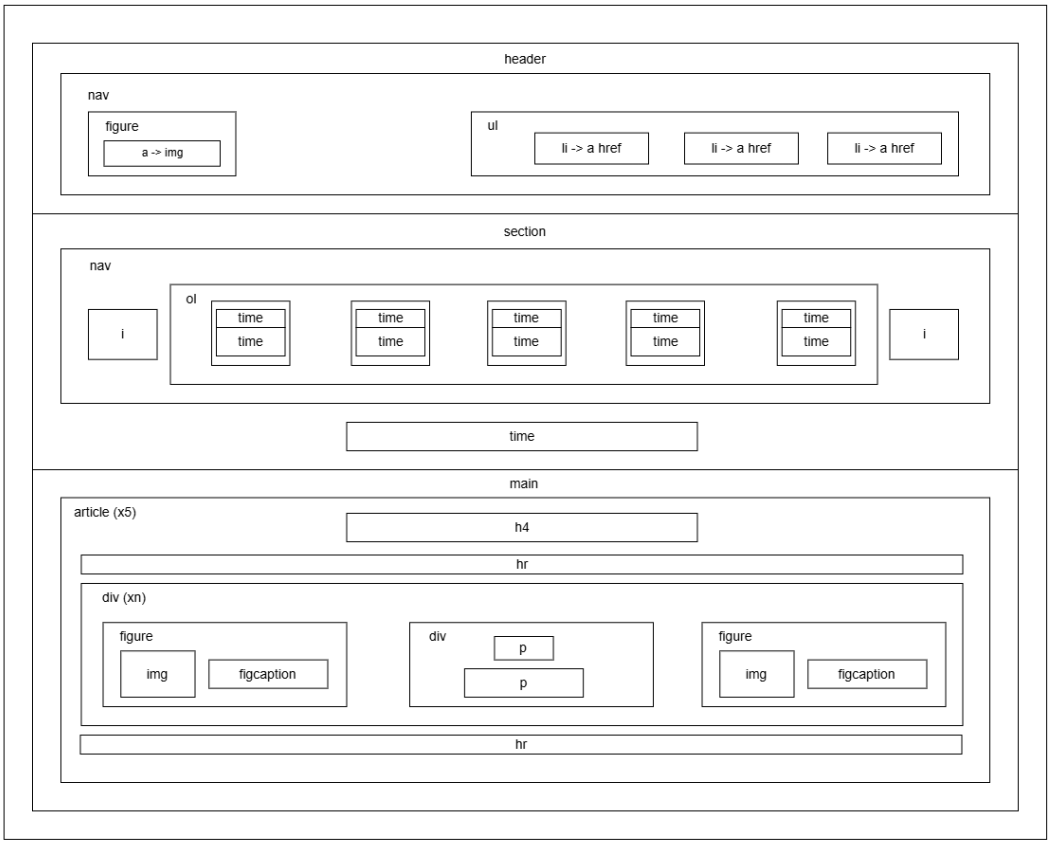


Figura 39: Esquema CSS de la página de calendario

7.5.1. Responsividad

Se aplica una media query con punto de ruptura en 768px, que ajusta el tamaño de los elementos del carrusel, oculta los nombres de los equipos (dejando solo los escudos visibles) y alinea correctamente las imágenes (figuras 40 y 41).

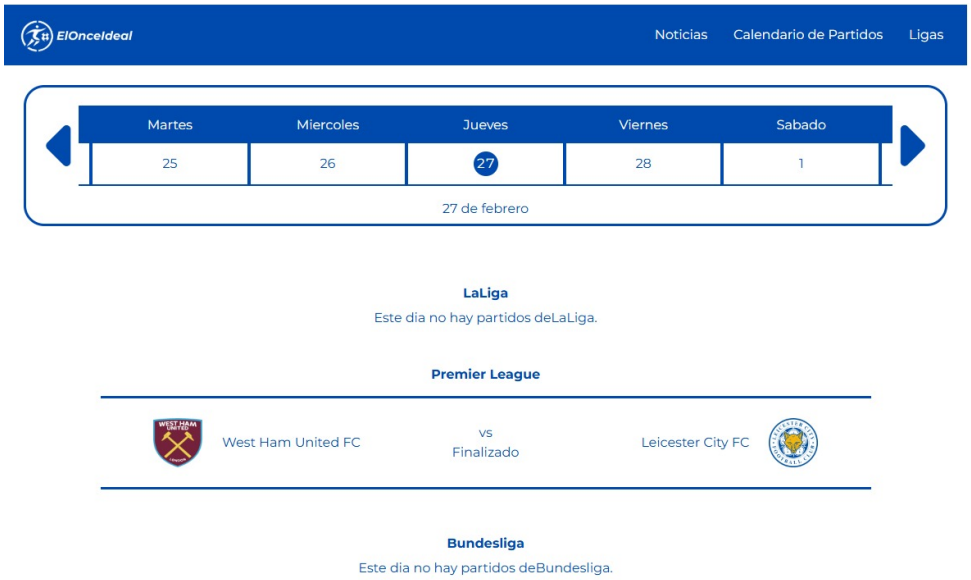


Figura 40: Responsividad: Página de calendario vista en ordenador



Figura 41: Responsividad: Página de calendario vista en móvil

7.6. CSS Grid: Página de partido

En la página de partido, se hace uso de CSS Grid para organizar las secciones informativas, como alineaciones, resultado, y ficha del encuentro. Esta técnica permite crear una cuadrícula con control total sobre filas y columnas. El esquema que se asocia a esta página es el mostrado en la figura 42.

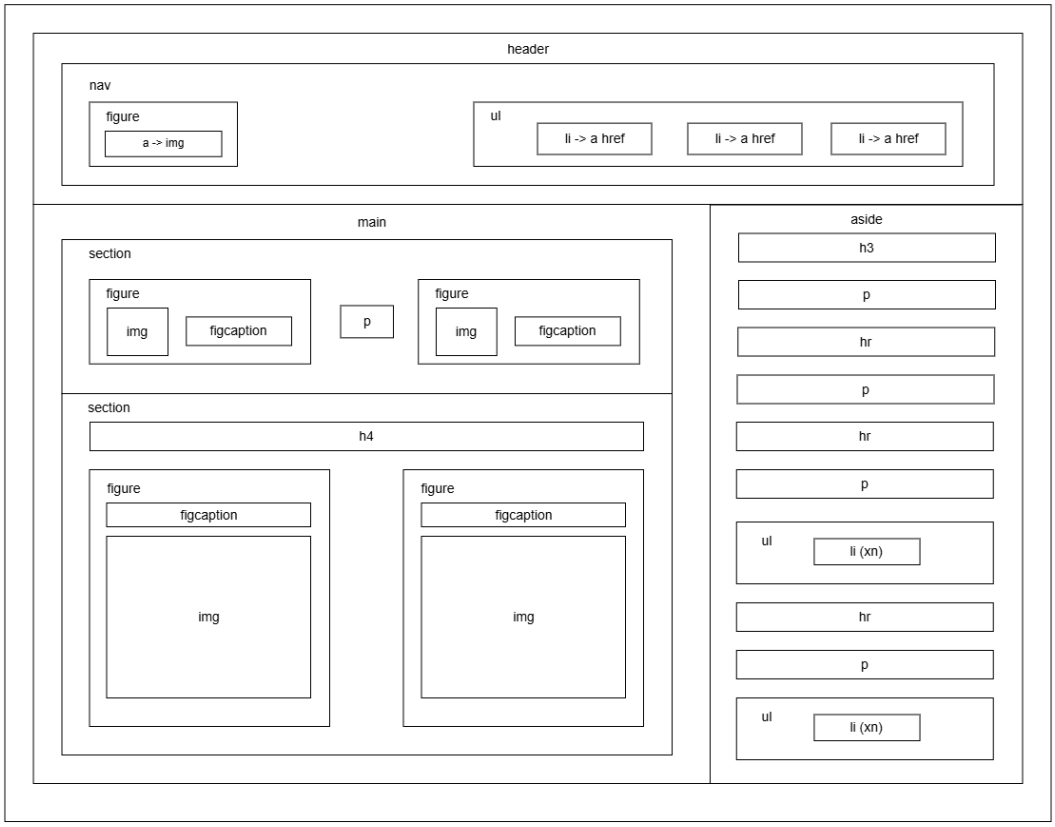


Figura 42: Esquema CSS de la página de partido

7.6.1. Responsividad

Con una media query con punto de ruptura en 768px, se reorganizan las columnas para que cada sección (resultado, alineaciones y ficha) ocupe el ancho completo del contenedor. También se ocultan elementos como los nombres de los equipos para una vista más limpia en móvil (figuras 43 y 44).



Figura 43: Responsividad: Página de partido vista en ordenador

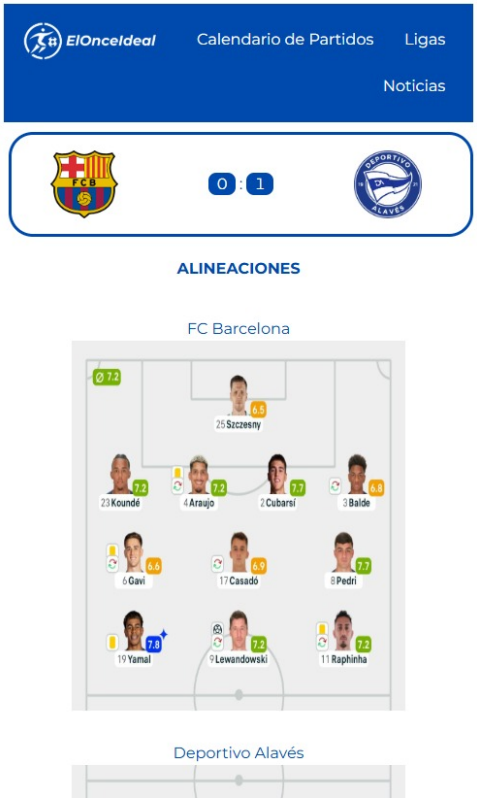


Figura 44: Responsividad: Página de partido vista en móvil

7.7. Librería Bootstrap: Página de ligas

Para la página de ligas, se ha integrado la librería Bootstrap, aprovechando sus clases y sus utilidades de responsividad para crear una segunda barra de navegación y distribuir los contenidos. El esquema correspondiente se recoge en la figura 45.

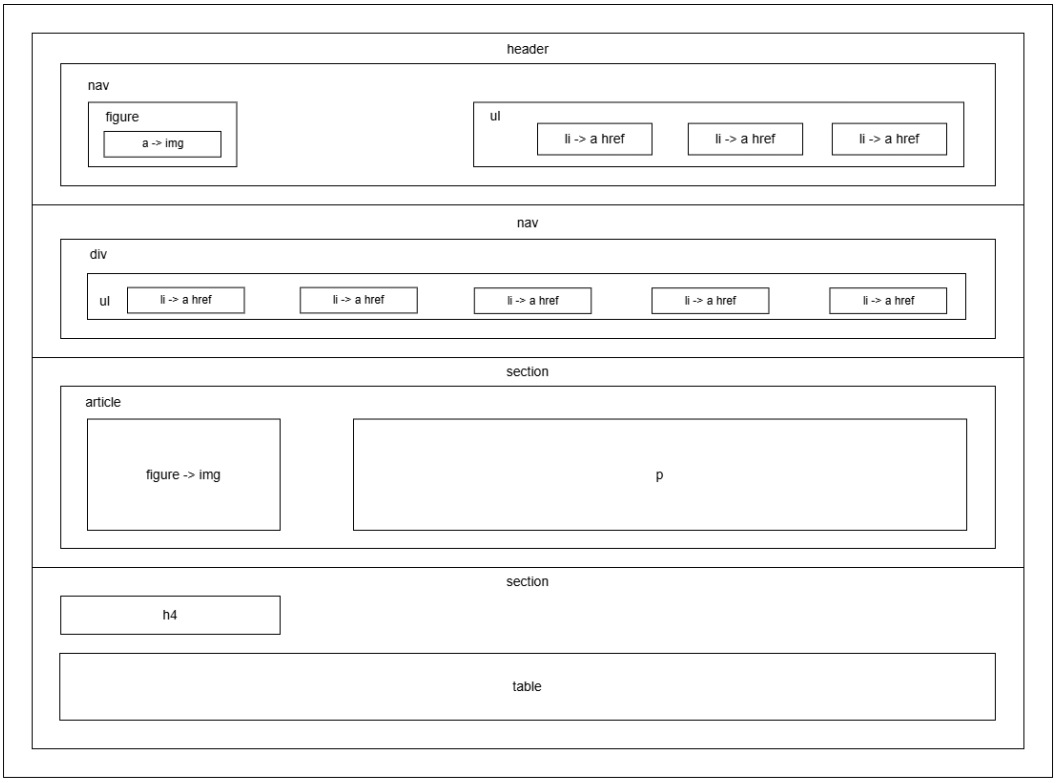


Figura 45: Esquema CSS de la página de ligas

7.7.1. Responsividad

Bootstrap incluye puntos de ruptura predefinidos (768px para md) que reorganizan los elementos y ajustan sus proporciones según el ancho del dispositivo (figuras 46 y 47).



	Equipo	PJ	V	E	D	GF	GC	DG	PTS
1	Barcelona	29	21	3	5	82	28	59	66
2	Real Madrid	29	19	6	4	62	29	33	63
3	Atlético Madrid	29	16	9	4	47	23	24	53

Figura 46: Responsividad: Página de ligas vista en ordenador



Figura 47: Responsividad: Página de ligas vista en móvil

8. JavaScript

8.1. Semana 1: DOM

8.1.1. Noticias.js

El objetivo será darle funcionalidad al "botón AMPLIAR"de las secciones de noticias, de tal forma, que al pulsar sobre ellos se incorpore un nuevo conjunto de noticias a la sección. La figura 48 muestra el estado inicial de la sección de noticias, al pulsar sobre el botón vemos que se incorporan cuatro nuevas noticias (figura 49).



Figura 48: Estado inicial de una sección de noticias



Figura 49: Estado posterior al pulsar AMPLIAR

Para lograrlo se diseña la función de la figura 50 que sigue los pasos:

1. Seleccionar la sección donde se incorporarán las nuevas noticias. Se emplean las funciones de acceso al DOM **getAttribute()** y **getElementById()**.

2. Crear el div que contendra el nuevo conjunto de noticias(figures), usa la función de acceso al DOM **createElement()**.

3. Obtener los datos de las siguientes noticias a través de la función *proximasNoticias()*. Dicha función devuelve en esta fase de JavaScript un array de cuatro objetos iguales

que representan el contenido de cada noticia; en la 3º fase de JavaScript (acceso a datos en archivos) se empleara proximasNoticias() para leer las noticias de un archivo.

4. Crear las *figure* de las noticias e incorporarlas al div previamente creado. Se usa las funciones de acceso al DOM **appendChild()**.

5. Insertamos el div con las figuras en la sección.

```
var evento = function(b) {
        var id = this.getAttribute("data-id");
        var seccion = document.getElementById(id);

        var div = document.createElement("div");

        var noticiasObj;
        if(id == "ultimas_noticias") {
           noticiasObj = proximasNoticias(id, contadorUltimasNoticias, this);
            contadorUltimasNoticias += noticiasObj.length;
        } else if (id == "mercado_fichajes") {
            noticiasObj = proximasNoticias(id, contadorMercadoFichajes, this);
            contadorMercadoFichajes += noticiasObj.length;
        }
        div.appendChild(crearFigure("./../Imagenes/Noticias/" + noticiasObj[0].imagen, noticiasObj[0].titular, noticiasObj[0].enlace));
        div.appendChild(crearFigure("./../Imagenes/Noticias/" + noticiasObj[1].imagen, noticiasObj[1].titular, noticiasObj[1].enlace));
        div.appendChild(crearFigure("./../Imagenes/Noticias/" + noticiasObj[2].imagen, noticiasObj[2].titular, noticiasObj[2].enlace));
        div.appendChild(crearFigure("./../Imagenes/Noticias/" + noticiasObj[3].imagen, noticiasObj[3].titular, noticiasObj[3].enlace));
        seccion.insertBefore(div, seccion.querySelector("p"));
}
```

Figura 50: Código evento de noticias.js

La función anteriormente descrita se activa al hacer click sobre los "botones AMPLIAR"(figura 51).

```
var botones = document.querySelectorAll(".link-paragraph a");

var addListener = function(b) {b.addEventListener("click", evento)};

botones.forEach(addListener);
```

Figura 51: Noticias Listener

8.1.2. Calendario.js

El objetivo será darle funcionalidad al carrusel de fechas, de tal forma que se desplacen los dias lateralmente y se modifique el contenido de la página mostrando los partidos que corresponden al dia central del carrusel. En la figura 52 aparece el estado inicial del carrusel de fechas; tras desplazarlo a la derecha vemos que los elementos se han desplzado una posición en la dirección mencionada, ocultando el elemento más a la derecha y apareciendo una nueva fecha por la izquierda, además se han eliminado los partidos de la página y cargado unos nuevos.



Figura 52: Estado inicial del carrusel

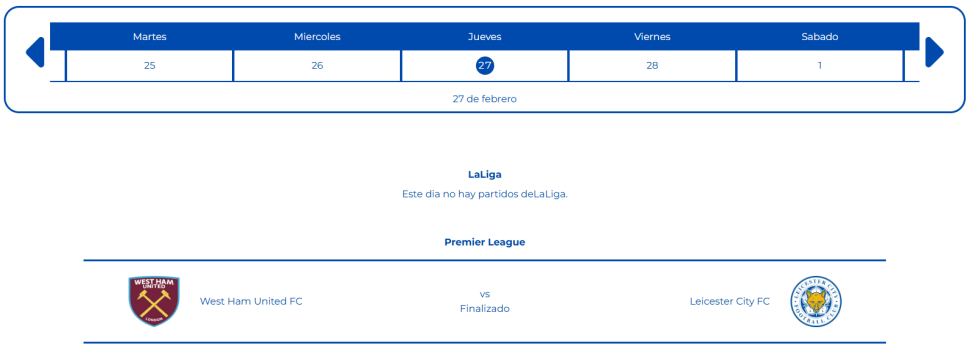


Figura 53: Estado del carrusel desplazado a la derecha

Para lograr este funcionamiento se diseña la función de la figura 54 que sigue los pasos:

1. **Inicialización y selección de elementos**. Se obtiene la *lista* de elementos del carrusel y el elemento central. Para lo que se usa las funciones de acceso al DOM **getElementeById()** y **querySelectorAll()**.

2. **Preparación del elemento central actual**. Se toma el elemnto central del carrusel y se elimina el "span"que lo resalta. Se usa la función de acceso al DOM **getElementsByTagName()**.

3. **Desplazamiento**. Se toma un elemento de un extremo del carrusel y se situa en el otro, posteriorment se actualiza la fecha del elemento desplazado. El extremo del que se toma el elemento que se cambia de extremo, depende del parametro *dirección*. Se usan las funciones de acceso al DOM **getAttribute()**, **querySelector()**, **removeChild()**, **appendChild()** y **setAttribute()**.

4. **Actualizar detalles de la interfaz del carrusel**. Se coloca el span al nuevo elemento central para resaltarlo y se actualiza el parrafo debajo del carrusel que muestra la fecha actual.

5. **Cargar Partidos**. Se cargan los partidos en las secciones correspondientes con la función *incluirPartidos()*, en esta fase de JavaScript incluirPartidos() carga información por defecto; en la 3º fase de JavaScript (lectura de información desde archivos) se cargarán distintos Partidos de un documento json.

```
var evento = function (direccion) {
    var carrusel = document.getElementById("carrusel_fechas");
    var lista = carrusel.querySelectorAll("li");
    var central = lista.item(lista.length / 2);


    var oldSpanText = central.getElementsByTagName("span")[0].textContent;
    central.lastElementChild.innerHTML = oldSpanText;

    var proximoCentral
    var fechaElemento;
    var fecha;

    var ol = carrusel.querySelector("ol");
    var fechaSalto;
    var li;
    if (direccion > 0) {
        proximoCentral = lista.item(lista.length / 2 - 1);
        fechaElemento = proximoCentral.lastElementChild.getAttribute("datetime");
        fecha = new Date(fechaElemento);

        li = lista[lista.length -1];
        console.log(li);
        ol.removeChild(li);
        ol.insertBefore(li, ol.firstChild);

        fechaSalto = new Date(fecha);
        fechaSalto.setDate(fechaSalto.getDate() - 4);
```

(a) Primera parte de la función

```
    } else {
        proximoCentral = lista.item(lista.length / 2 + 1);
        fechaElemento = proximoCentral.lastElementChild.getAttribute("datetime");
        fecha = new Date(fechaElemento);

        li = ol.querySelector("li");
        console.log(li);

        ol.removeChild(li);
        ol.appendChild(li);


        fechaSalto = new Date(fecha);
        fechaSalto.setDate(fechaSalto.getDate() + 4);
    }

    console.log(fechaSalto);
    li.lastElementChild.innerHTML = fechaSalto.toLocaleDateString('es-ES', { day: 'numeric' });
    li.lastElementChild.setAttribute("datetime", fechaSalto.toISOString().split('T')[0]);
    li.firstElementChild.innerHTML = fechaSalto.toLocaleDateString('es-ES', { weekday: 'long' });

    var text = proximoCentral.lastElementChild.textContent;
    proximoCentral.lastElementChild.innerHTML = "<span class=\"seleccionado\">" + text + "</span>";
    var opciones = { day: 'numeric', month: 'long' };
    var fechaTexto = fecha.toLocaleDateString('es-ES', opciones);
    document.getElementById("fecha_actual").innerHTML = fechaTexto;

    var ligas = ["LaLiga", "Premier League", "Bundesliga", "Serie A", "Ligue 1"];
    ligas.forEach(function(liga) {
        incluirPartidos(fecha.toISOString().split('T')[0], liga);
    });
}
```

(b) Segunda parte de la función

Figura 54: Código de Calendario.js

La página calendario contará con tres tipos de eventos (figura 55):

- **Click**. Al pulsar sobre las flechas se desplaza el carrusel.
- **Keypress**. Si pulsamos las teclas "d.º flecha derecha, el carrusel se desplza hacia la derecha, si pulsamos .ª.º flecah izquierda el carrusel se desplazará hacia la izquierda.
- **Load**. Carga los partidos iniciales de la página.

```
left_btn.addEventListener("click", function() {
    evento(-1);
});

right_btn.addEventListener("click", function() {
    evento(1);
});
document.addEventListener("keydown", function (event) {
    if (event.key === "ArrowLeft" || event.key == "a" || event.key == "A") {
        left_btn.click();
    }

    if (event.key === "ArrowRight" || event.key == "d" || event.key == "D") {
        right_btn.click();
    }
});

window.addEventListener("load", function () {
    var fechaElemento = document.getElementsByTagName("span")[0].parentElement.getAttribute("datetime");
    var fecha = new Date(fechaElemento);
    var ligas = ["LaLiga", "Premier League", "Bundesliga", "Serie A", "Ligue 1"];
    ligas.forEach(function(liga) {
        incluirPartidos(fecha.toISOString().split('T')[0], liga);
    });

});
```

Figura 55: Calendario Eventos

8.2. Semana 2: JQuery y JES6

8.2.1. JQuery (noticas.js)

Se modifica la función de noticias.js para que empleé jQuery (figura **??**), los cambios destacables son:

- **Selección de elementos**. Se selecciona la sección empleando el selector del constructor de JQuery, mediante un id tomado previamente con la función *attr()*.

```
var id = $(this).attr("data-id");
var seccion = $("#" + id);
```

- **Creación y Manipulación de elementos**. El elemento div ahora se crea con Jquery. Se oculta con *hide()* para agregar una animación posteriormente.

```
var div = $("<div>").hide();
```

- **Manipulacción del DOM**. Las figuras se agregan al div anterior con *append()* y el div se agrega antes del único parrafo de la sección con la función *before()*. El parrafo fue tomado previemente con *find().first()*.

```
div.append(figure);
seccion.find("p").first().before(div);
```

- **Animaciones**. Se agrega un efecto de slide al nuevo contenedor.

```
div.slideDown(500);
```

```
var evento = function(b) {
        var id = $(this).attr("data-id");
        var seccion = $("#" + id);

        var div = $("<div>").hide();

        var noticiasObj;
        if(id == "ultimas_noticias") {
           noticiasObj = proximasNoticias(id, contadorUltimasNoticias, this);
            contadorUltimasNoticias += noticiasObj.length;
        } else if (id == "mercado_fichajes") {
            noticiasObj = proximasNoticias(id, contadorMercadoFichajes, this);
            contadorMercadoFichajes += noticiasObj.length;
        }
        noticiasObj.forEach(function(noticia) {
            var figure = crearFigure("./../Imagenes/Noticias/" + noticia.imagen, noticia.titular, noticia.enlace);
            div.append(figure);
        });

        seccion.find("p").first().before(div);

        div.slideDown(500);
}
```

Figura 56: Noticias - código JQuery

8.2.2. JES6 (calendario.js)

Se modifica la función de calendario.js para que empleé JES6 (figura 57), los cambios destacables son:

- **Declaración let y const**. let reemplaza a var con scope de bloque y const para variables que no se reasignan.

```
let fecha;
const ol=carrusel.querySelector("ol");
```

- **Funciones flecha**. Aporta una sintaxis más concisa para funciones anónimas.

```
lista.forEach(element => {
    ...}
```

- **Bucles for...of**. Usado para iterar sobre los valores de ligas.

```
for(liga of ligas) {
    ...}
```

- **Uso de promesas**. Se incorporo nuevo código que anima el desplazamiento del carrusel mediante un translateX. Para esperar a la finalización de dichas animaciones se emplean promesas:

```
Promise.all(animaciones).finally(...)
```

```
let evento = function (direccion) {
    let carrusel = document.getElementById("carrusel_fechas");
    let lista = carrusel.querySelectorAll("li");
    let central = lista.item(lista.length / 2);

    let oldSpanText = central.getElementsByTagName("span")[0].textContent;
    central.lastElementChild.innerHTML = oldSpanText;

    lista.forEach(element => {
        let w = element.clientWidth;

        element.style.transition = "1s";
        element.style.transform = "translateX(" + (direccion * w) + "px)";

        incluirAnimacion(element);
    });

    Promise.all(animaciones).finally(() => {
        animaciones = [];
        let proximoCentral
        let fechaElemento;
        let fecha;

        const ol = carrusel.querySelector("ol");
        let fechaSalto;
        let li;
        if (direccion > 0) {
            proximoCentral = lista.item(lista.length / 2 - 1);
            fechaElemento = proximoCentral.lastElementChild.getAttribute("datetime");
            fecha = new Date(fechaElemento);

            li = lista[lista.length - 1];
            console.log(li);
            ol.removeChild(li);
            ol.insertBefore(li, ol.firstChild);

            fechaSalto = new Date(fecha);
            fechaSalto.setDate(fechaSalto.getDate() - 4);
```

(a) Primera parte de la función

```
        } else {
            proximoCentral = lista.item(lista.length / 2 + 1);
            fechaElemento = proximoCentral.lastElementChild.getAttribute("datetime");
            fecha = new Date(fechaElemento);

            li = ol.querySelector("li");
            console.log(li);

            ol.removeChild(li);
            ol.appendChild(li);


            fechaSalto = new Date(fecha);
            fechaSalto.setDate(fechaSalto.getDate() + 4);
        }

        li.lastElementChild.innerHTML = fechaSalto.toLocaleDateString('es-ES', { day: 'numeric' });
        li.lastElementChild.setAttribute("datetime", fechaSalto.toISOString().split('T')[0]);
        li.firstElementChild.innerHTML = fechaSalto.toLocaleDateString('es-ES', { weekday: 'long' });

        let text = proximoCentral.lastElementChild.textContent;
        proximoCentral.lastElementChild.innerHTML = "<span class=\"seleccionado\">" + text + "</span>";
        const opciones = { day: 'numeric', month: 'long' };
        const fechaTexto = fecha.toLocaleDateString('es-ES', opciones);
        document.getElementById("fecha_actual").innerHTML = fechaTexto;

        const ligas = ["LaLiga", "Premier League", "Bundesliga", "Serie A", "Ligue 1"];
        for(liga of ligas) {
            incluirPartidos(fecha.toISOString().split('T')[0], liga);
        }

        lista.forEach(element => {
            let w = element.clientWidth;
            element.style.transition = "0s";
            element.style.transform = "translateX(0px)";

        });

    });
}
```

(b) Segunda parte de la función

Figura 57: Calendario - código ES6

Además de lo anterior se modifican los *addEventListener* para que empleén funciones flecha (figura 58).

```
left_btn.addEventListener("click", () => evento(-1));

right_btn.addEventListener("click", () => evento(1));

document.addEventListener("keydown", (event) => {
    if (event.key === "ArrowLeft" || event.key == "a" || event.key == "A") {
        left_btn.click();
    }

    if (event.key === "ArrowRight" || event.key == "d" || event.key == "D") {
        right_btn.click();
    }
});

window.addEventListener("load", () => {
    let fechaElemento = document.getElementsByTagName("span")[0].parentElement.getAttribute("datetime");
    let fecha = new Date(fechaElemento);
    const ligas = ["LaLiga", "Premier League", "Bundesliga", "Serie A", "Ligue 1"];
    ligas.forEach( liga => incluirPartidos(fecha.toISOString().split('T')[0], liga));
    clickableDiv();
});
```

Figura 58: Eventos Calendario - ES6

8.3. Semana 3: Carga de contenido

8.3.1. Carga de XML con fetch (Noticias.js)

Las noticias que se desplegaran al pulsar el "botón AMPLIAR"provienen del fichero *noticias.xml* almacenado en el directorio data. Se carga el contenido de dicho fichero

mediante la función *cargarXML*(figura 59), la cual, emplea la **API Fetch**. Mediante una cadena de promesas primero convierte la respuesta a texto (response.text()) y luego parsea el texto a XML usando DOMParser. DOMParser es un objeto nativo del navegador para convertir strings a documentos XML/HTML.

```
function cargarXML() {
    fetch("../data/noticias.xml")
    .then(response => response.text())
    .then(str => {
        let parser = new DOMParser();
        xmlDoc = parser.parseFromString(str, "text/xml");
    })

}
```

Figura 59: Noticias - Cargar XML

Posteriormente el objecto xmlDoc (un objeto DOMParser) es empleado en la función **proximasNoticias()**(figura 60 la cual lee el contenido de las noticias en función de la sección que se desee ampliar y devuelve una lista de objetos con los datos leídos de las noticias.

```
function proximasNoticias(seccion, id, btn) {
    let sec = xmlDoc.getElementsByTagName(seccion)[0];
    let proximasNoticias = [];
    let contador = 0;
    let noticias = sec.getElementsByTagName("noticia");
    for(e of noticias) {
        if(parseInt(e.getAttribute("id")) >= id){
            let enlace = e.getElementsByTagName("enlace")[0].innerHTML;
            let imagen = e.getElementsByTagName("imagen")[0].innerHTML;
            let titular = e.getElementsByTagName("titular")[0].innerHTML;
            proximasNoticias.push({enlace, imagen, titular});
            contador++;
        }
        if(contador >= 4) {
            break;
        }
    }

    if( (contador + id) == noticias.length) {
        btn.parentElement.classList.add("link-disabled");
        btn.parentElement.classList.remove("link-paragraph");
        btn.removeEventListener("click", evento);
    }

    return proximasNoticias;
}
```

Figura 60: Noticias - Usar XML

8.3.2. Carga de JSON con XMLHttpRequest (Calendario.js)

La lista de partidos que se muestran al cargar la página e interactuar con el carrusel se obtienen del archivo *partidos.json* en el directorio data. Se carga el contenido de dicho fichero mediante la función *leerJSON()*(figura 61, la cual, emplea **XMLHttpRequest**. Primero se crea una petición GET para obtener el JSON y se configura el callback en *onreadystatechange*, donde se parsea el JSON convirtiendo la respuesta de texto a objeto JS con **JSON.parse()**.

```
async function leerJSON() {
    xhr = new XMLHttpRequest();
    xhr.open("GET", "../data/partidos.json", true);
    xhr.onreadystatechange = function () {
        if (xhr.readyState == 4 && xhr.status == 200) {
            data = JSON.parse(xhr.responseText);
        }
    };
    xhr.send();
}
```

Figura 61: Calendario - Cargar JSON

Posteriormente el objeto data (JSON parseado), se emplea en la funcion **incluirPartidos()**(figura 62) que extrae la información de los partidos para la liga y fecha especificados, construye los elementos del DOM que representan los partidos y los incorpora en la sección correspondiente.

```
function incluirPartidos(strFecha, liga) {
    let art = document.getElementById(liga);

    art.querySelectorAll("p").forEach(d => d.remove());
    art.querySelectorAll("div").forEach(d => d.remove());
    art.querySelectorAll("hr").forEach(h => h.remove());

    if (data["partidos"][strFecha] == undefined) {
        art.appendChild(generarNoPartidoElement(liga));
        return;
    }

    if (data["partidos"][strFecha][liga] == undefined) {
        art.appendChild(generarNoPartidoElement(liga));
        return;
    }

    if (data["partidos"][strFecha][liga].length > 0) {
        let hr = document.createElement("hr");
        art.appendChild(hr);
    }

    for (i of data["partidos"][strFecha][liga]) {
        let ficha = generarFichaPartido(i["equipo_local"], i["imagen_local"], i["equipo_visitante"], i["imagen_visitante"]);
        art.appendChild(ficha);
        let hr = document.createElement("hr");
        art.appendChild(hr);
    }
}
```

Figura 62: Calendario - Usar JSON